4 - Mettre en place une API REST avec Express

Astuces :

- on utilise la méthode « get » pour récupérer une requête http.

Installez le framework Express

Express est un framework web Node.Js minimal et flexible qui offre un ensemble robuste de fonctionnalités pour les applications web et mobiles. Il fournit un ensemble de méthodes permettant de traiter les requêtes HTTP et fournit des middlewares pour étendre ses fonctionnalités.

Installation du module :
Créez un répertoire « server » et rendez-vous à la racine de ce répertoire dans le terminal puis tapez les commandes :
npm init -y
npm install express

Mettez en place un serveur Express simple

Créez un fichier **server.js** qui sera le point d'entrée de l'application Express. Dans ce fichier, placez le code suivant :

```
const express = require('express');
const app = express();
const port = 8000;

app.listen(port, () => {
    console.log('Server app listening on port ' + port);
});
```

La méthode **express()** permet de déclarer l'application. On affecte le résultat de cette méthode dans la constante **app** et on spécifie le port d'écoute que l'on désire dans **port**.
À partir de cet objet **app**, nous disposons de plusieurs méthodes. L'une d'entre elles est la méthode **listen()**. Elle est utilisée pour lier et écouter les connexions sur l'hôte et le port spécifiés. Elle est identique à la méthode **http.Server.listen()** de Node. Si le numéro de port est omis ou est égal à 0, le système attribuera un port arbitraire non utilisé. En voici la syntaxe :
app.listen([port[, host[, backlog]]][, callback])

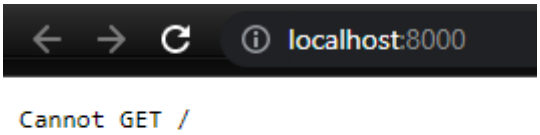
Explication de la syntaxe

- **port** (facultatif). Il spécifie le port sur lequel notre application écoute.
- **host** (facultatif). Il spécifie l'adresse IP de l'hôte sur lequel écoute notre application. Vous pouvez spécifier l'hôte si et seulement si vous avez déjà indiqué le port, comme vous pouvez le voir dans la syntaxe ci-dessus, cela signifie que le port doit être mentionné avant de spécifier host et backlog.
- **backlog** (facultatif). Il spécifie la longueur maximale de la file d'attente des connexions en attente. Vous pouvez spécifier la file d'attente si et seulement si vous avez déjà spécifié le port et l'hôte.
- **callback** (facultatif). Il spécifie une fonction qui sera exécutée une fois que votre application commence à écouter le port spécifié. Vous pouvez spécifier la fonction callback seule, c'est-à-dire sans spécifier le port.

L'application renvoyée par **express()** est en réalité une fonction JavaScript, conçue pour être transmise aux serveurs HTTP de Node en tant que callback pour traiter les demandes. Cela permet de fournir facilement les versions HTTP et HTTPS de votre application avec la même base de code, car l'application n'hérite pas de ces dernières.

Démarrer le serveur Express

Dans le terminal, tapez la commande :
npm run start
Entrez l'URL de votre serveur dans votre navigateur : http://localhost:8000



Votre serveur n'est pas encore en mesure d'envoyer une réponse pour les URLs demandées, c'est la raison pour laquelle vous pouvez voir ce message dans le navigateur. Vous verrez plus loin comment gérer les routes d'une application Express.

Pour aller plus loin : la documentation Express

Lisez la documentation d'express pour plus de détails.

Grâce à la méthode express(), vous pouvez déclarer une application Express et utiliser la méthode listen() pour lier et écouter les connexions sur l'hôte et le port spécifiés.

Vous démarrerez le serveur avec la commande npm run start

Créez une application Express from scratch

Initialisation du projet

Créez un répertoire **express_app** pour votre projet et, à l'intérieur, créez un fichier **app.js** puis initialiser le projet avec la commande **npm init**. Assurez-vous que vous avez **app.js** comme point d'entrée (**entry point**) dans le fichier package.json.

Installation d'express

On installe express avec l'option save :
npm install express –save

L'application basique

Dans le fichier **app.js**, déclarez l'application express et la méthode pour lancer le serveur de l'application sur un port déterminé :

```
app.js > ...
1   const express = require('express');
2   const app = express();
3   const port = 3000;
4
5
6   app.listen(port, () => {
7       console.log(`l'Application écoute sur le port ${port}!`)
8   });
9
```

Le gestionnaire de routes

Nous allons maintenant faire en sorte de pouvoir gérer les routes de manière à ce que notre serveur puisse commencer à répondre aux requêtes entrantes. Pour faire simple, nous commencerons par une route en GET de manière à prendre en compte une requête simple provenant d'un navigateur. Ajoutez une fonction de gestion de route **get()** pour les requêtes HTTP GET vers la racine du site **localhost :3000/.** La fonction callback du gestionnaire de route prend deux arguments **req** et **res** (la requête reçue et la réponse à envoyer).

Ici, vous utilisez la méthode **send()** de l'objet **res** pour renvoyer la chaîne de caractères "Hello world!" lorsqu'un utilisateur visite la page principale (racine) de votre site avec son navigateur. Si vous souhaitez envoyer une réponse au format JSON plutôt qu'une réponse au format texte, alors vous pourrez utiliser **res.json().** Si vous souhaitez envoyer un fichier, optez pour la méthode **res.sendFile().**
Il existe des gestionnaires de route pour tous les autres **verbes Http (GET, POST, …)**. Les fonctions de routes achèvent le cycle requête-réponse HTTP en renvoyant une réponse au client. Vous pouvez rendre un fichier html comme réponse avec la méthode **res.render().**

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
    res.send('Hello World!')
});

app.listen(port, () => {
    console.log(`l'Application écoute sur le port ${port}!`)
});
```

Utilisation des middlewares

Une application Express se compose d'une série de fonctions appelées **middlewares (intergiciels**). Elles peuvent lire les requêtes et réponses, les analyser et les manipuler. Un middleware Express reçoit la méthode **next()**, qui fait passer l'exécution au middleware suivant (un autre middleware ou gestionnaire de route).
Vous pouvez écrire vos propres middlewares ou utiliser les méthodes d'un module tiers. Pour ajouter un middleware, on utilise **app.use()**.

```
const express = require('express')
const app = express()
const port = 3000

app.get( '/' ,(req, res) => {
    res.type( 'text/plain' )
    res.send( 'Server Expresso   Bienvenue sur mon appli express' )
})
app.get( '/about', ( req, res ) => {
    res.type( 'text/plain' )
    res.send('Je suis une application express basique')
})
app.use( ( req, res ) => {
    res.type( 'text/plain' )
    res.status( 404 )
    res.send('404 page non trouvée')
})

app.listen(port, () => {
    console.log(`l'Application écoute sur le port ${port}!`)
})
```

Utilisation de nodemon

Si vous voulez coder dans votre projet, il faudra redémarrer le serveur à chaque modification, ça devient très vite ennuyant. Pour pouvoir bénéficier d'un **redémarrage automatique (hotreload) pour que vos modifications soient directement prises en compte**, alors utilisez **nodemon**.
npm install --save-dev nodemon
Au lieu de démarrer votre application avec **npm run start**, utilisez désormais la commande :
nodemon app.js

Vous pouvez également choisir de modifier le script 'start' dans le fichier package.json de manière à remplacer le contenu de la commande par cette nouvelle commande, de cette manière, ça ne changera pas vos habitudes.

Servir les fichiers statiques

Vous êtes actuellement en mesure de répondre à toute requête entrante et de renvoyer du HTML lorsqu'un utilisateur visite votre site. Par contre, vous n'avez encore jamais envoyé de fichiers image, JS ou CSS.
Pour prendre en charge ces fichiers statiques, nous allons avoir besoin de spécifier via Express, où aller chercher ces fichiers lorsqu'ils sont demandés.
Utilisez la ligne ci-dessous pour servir vos ressources statiques (css, images et JavaScripts) à partir d'un répertoire nommé « public » :
app.use(express.static('public')) ;

Si vous le souhaitez, vous pouvez tout à fait faire en sorte de servir plusieurs dossiers de contenu statiques, simplement en ajoutant un autre répertoire à express.static

Traitement des erreurs

Les erreurs sont traitées par des fonctions spéciales avec quatre arguments, les erreurs, la requête, la réponse et next pour passer au middleware suivant :
Elles doivent être **appelées après tous les autres app.use() et appels de routes**. Express intègre un gestionnaire d'erreur qui prend en charge toutes les erreurs restantes.

```
app.use(function(err, req, res, next) {
    console.error(err.stack);
    res.status(500).send("une erreur s'est produite !");
});
```

Générez des applications avec generator

Pour créer plus rapidement un squelette d'application, on utilise express-generator.
Avant de pouvoir utiliser express-generator, vous devrez l'installer sur votre machine de manière **globale** (pour y avoir accès dès que vous aurez besoin de créer une application Express).
Pour l'installer, tapez :
npm install express-generator -g

Une fois installé, et pour créer une application, entrez dans le terminal :
express mon_application
cd mon_application
npm install

- Sous MacOS ou Linux, exécutez : **DEBUG=mon_application:* npm start**
- Sous Windows, exécutez : **set DEBUG=mon_application:* & npm start**

Générez des applications avec generator

Pour créer plus rapidement un squelette d'application, on utilise express-generator.
Avant de pouvoir utiliser express-generator, vous devrez l'installer sur votre machine de manière **globale** (pour y avoir accès dès que vous aurez besoin de créer une application Express).
Pour l'installer, tapez :
npm install express-generator -g

Une fois installé, et pour créer une application, entrez dans le terminal :
express mon_application
cd mon_application
npm install

- Sous MacOS ou Linux, exécutez : **DEBUG=mon_application:* npm start**
- Sous Windows, exécutez : **set DEBUG=mon_application:* & npm start**

Pour aller plus loin : les middlewares

Consultez le Routing, l'écriture de middleware et l'utilisation des middlewares.

Je retiens : l'application Express

Une application Express peut être déclarée manuellement en initialisant un répertoire avec npm init et en installant le module express.

La méthode express() permet de déclarer l'application.

L'outil express-generator permet de créer un squelette d'application.

Créez une API REST

Dans ce cours, vous allez créer une API avec **express-generator** et utiliser une **base de données mongoDB.**

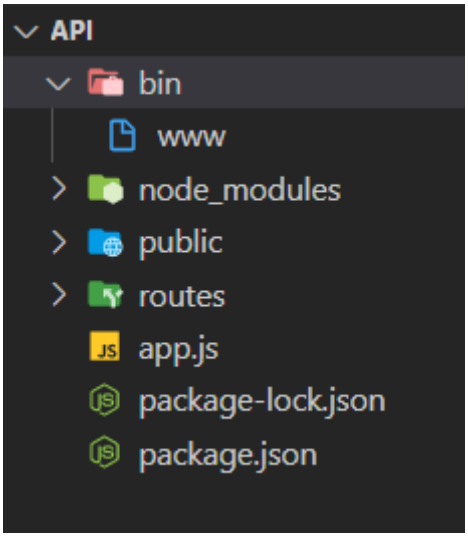
Si ce n'est pas déjà fait (cours précédent), installez express-generator en global sur votre machine **: npm install express-generator -g.**

Créez votre projet "api" avec la commande : **express api --no-view.** L'option no view permet de spécifier que nous ne souhaitons pas que express-generator crée les différentes vues associées, et c'est assez logique puisque nous allons créer une API.

Vous devriez obtenir un résultat similaire dans le terminal:

```
C:\Users\jeanm\Desktop>express api --no-view

   create : api\
   create : api\public\
   create : api\public\javascripts\
   create : api\public\images\
   create : api\public\stylesheets\
   create : api\public\stylesheets\style.css
   create : api\routes\
   create : api\routes\index.js
   create : api\routes\users.js
   create : api\public\index.html
   create : api\app.js
   create : api\package.json
   create : api\bin\
   create : api\bin\www

   change directory:
     > cd api

   install dependencies:
     > npm install

   run the app:
     > SET DEBUG=api:* & npm start
```

Entrez dans le répertoire depuis le terminal avec la commande : **cd api** et installez les dépendances avec **npm install.**
Voilà la structure du projet que vous devez obtenir :



Dans le répertoire **bin** se trouve le fichier **www,** qui appelle le fichier contenant l'application express **app.js** puis démarre le serveur.
Dans les scripts du fichier package.json, vous pouvez remarquer que la commande **npm start** est un alias de **node ./bin/www.** On peut donc installer le module **nodemon** et modifier ce script pour bénéficier du redémarrage automatique du serveur lorsqu'un changement dans le code de l'application intervient.
npm install nodemon

Ici, je l'ai installé dans les dépendances de production, il aurait été mieux de l'installer dans les dépendances de développement, mais ça ne change pas grand-chose pour notre projet de démonstration.

```
package.json > ...
1   {
2     "name": "api",
3     "version": "0.0.0",
4     "private": true,
      ▷ Déboguer
5     "scripts": {
6       "start": "nodemon ./bin/www"
7     },
8     "dependencies": {
9       "cookie-parser": "~1.4.4",
10      "debug": "~2.6.9",
11      "express": "~4.16.1",
12      "morgan": "~1.9.1",
13      "nodemon": "^2.0.20"
14    }
15  }
```

app.js

Le fichier app.js se présente de cette manière dans le squelette d'une application générée avec express-generator. On modifiera son comportement par la suite :

app.js > ...

```javascript
var express = require('express');
// import du module path module de chemin Node.js fournit un certain pour travaille
var path = require('path');
// import du module cookie-parser pour travailler avec les cookies
var cookieParser = require('cookie-parser');
// import du module morgan pour travailler avec les logs
var logger = require('morgan');
// import du fichier de route index.js
var indexRouter = require('./routes/index');
// import du fichier de route users.js
var usersRouter = require('./routes/users');

var app = express();
// Déclaration des middlewares utilisées par l'application
app.use(logger('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(cookieParser());
// On sert les ressources en déclarant le répertoire public
app.use(express.static(path.join(__dirname, 'public')));
// la racine de l'application utilisera la route déclarer dans index.js
app.use('/', indexRouter);
// la route /users utilisera les routes déclarées dans users.js
app.use('/users', usersRouter);
// On exporte l'application qui sera utilisée par le fichier www qui démarre le ser
module.exports = app;
```

Votre base mongoDB

Pour notre projet d'api, nous allons utiliser une base de données MongoDB. Avant de pouvoir utiliser ce service, vous allez devoir vous créer un compte gratuit sur leur site.
Rendez-vous sur **mongoDB** pour créer un compte, un **cluster Free Shared** et une base de données.

CLUSTERS > CREATE A SHARED CLUSTER

Create a Shared Cluster

Welcome to MongoDB Atlas! We've recomm
to your needs. For more information, check

Serverless

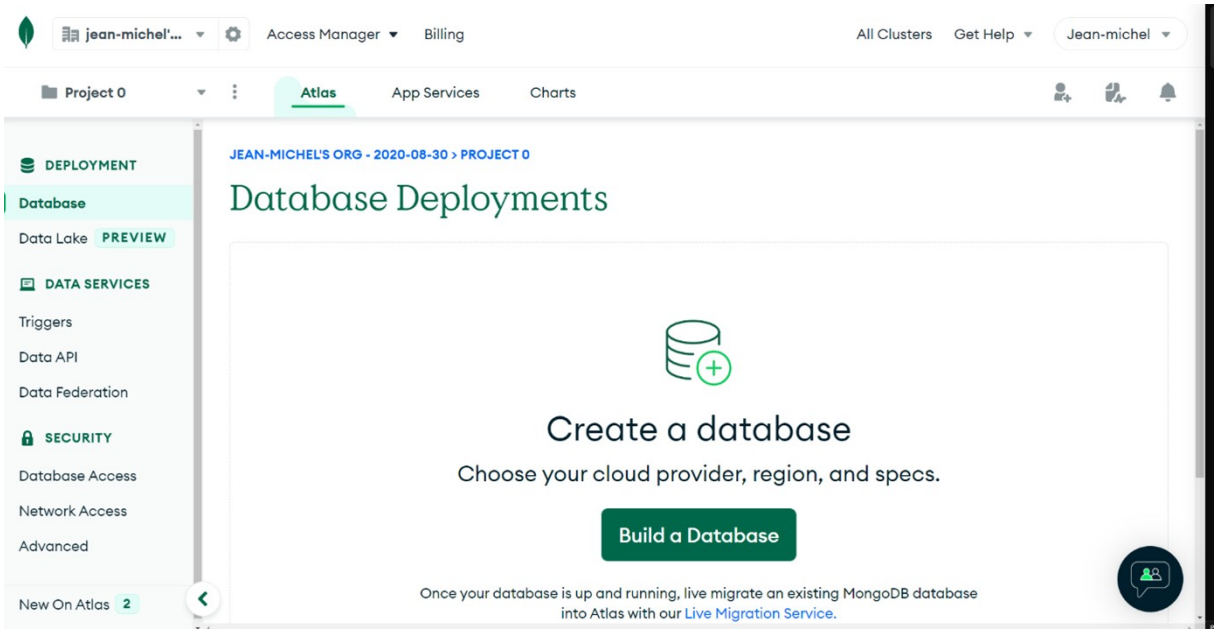
For learning and exploring MongoDB in a sar

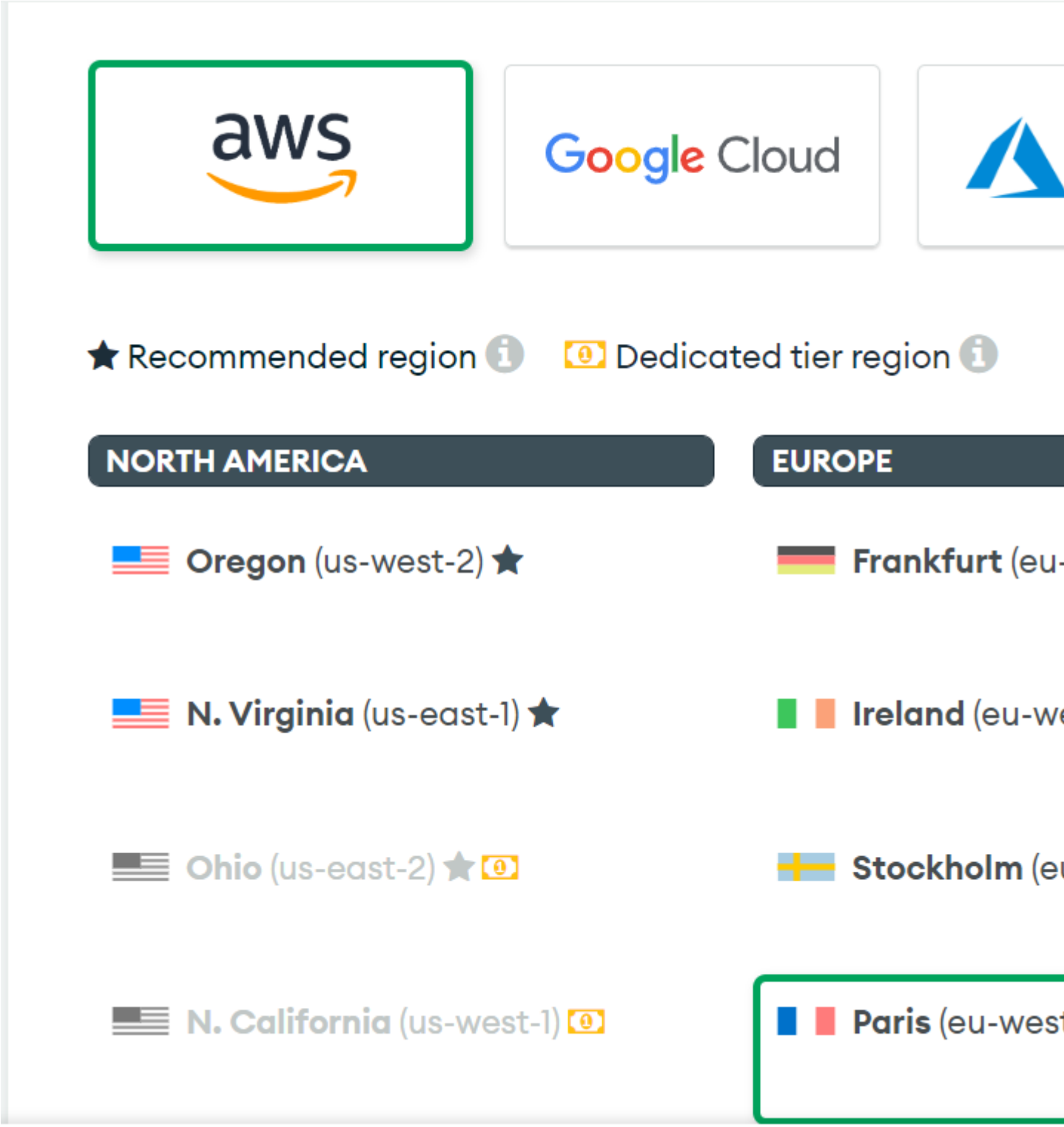
No credit card required to start. Upgrade to
Explore with sample datasets. Limit of one fr

Confidentialité - Conditions

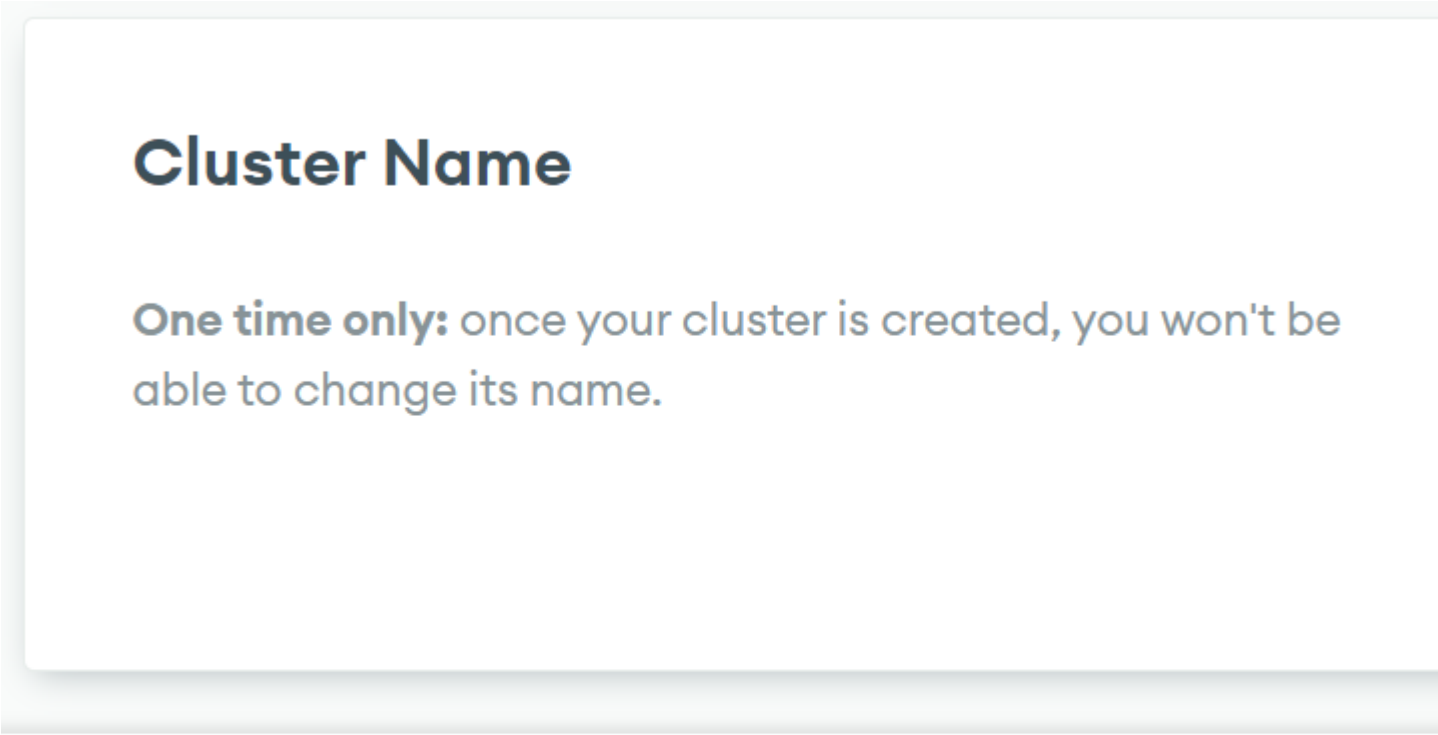
FREE

Free forever! Your M0 cluster is i
upgrade to a production cluster

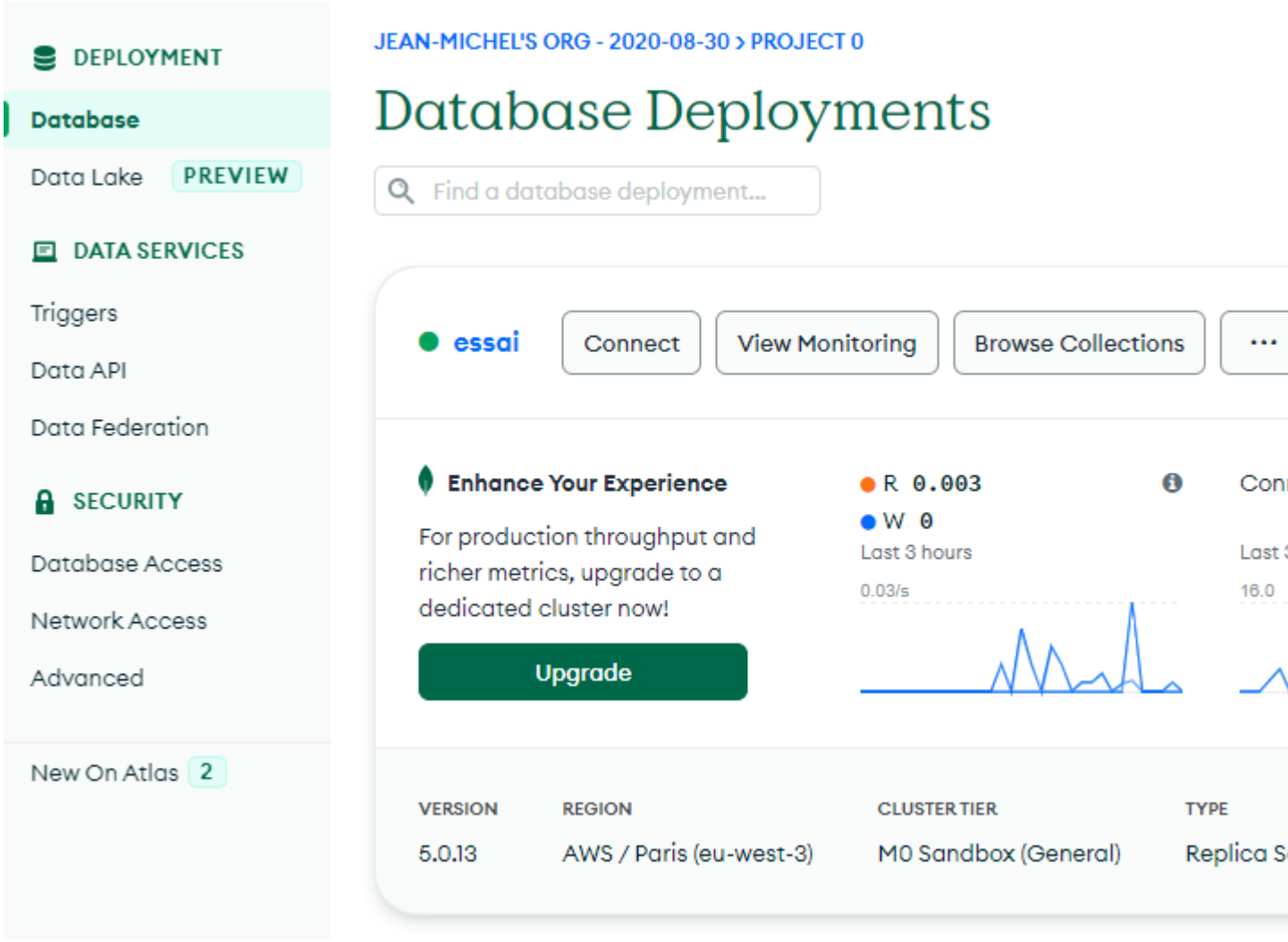

jean-michel'...
Access Manager
Billing
All Clusters
Get Help
Jean-michel
Project 0
Atlas
App Services
Charts
DEPLOYMENT
Database
Data Lake
PREVIEW
DATA SERVICES
Triggers
Data API
Data Federation
SECURITY
Database Access
Network Access
Advanced
New On Atlas
2
JEAN-MICHEL'S ORG - 2020-08-30 > PROJECT 0
Database Deployments
Create a database
Choose your cloud provider, region, and specs.
Build a Database
Once your database is up and running, live migrate an existing MongoDB database into Atlas with our Live Migration Service.



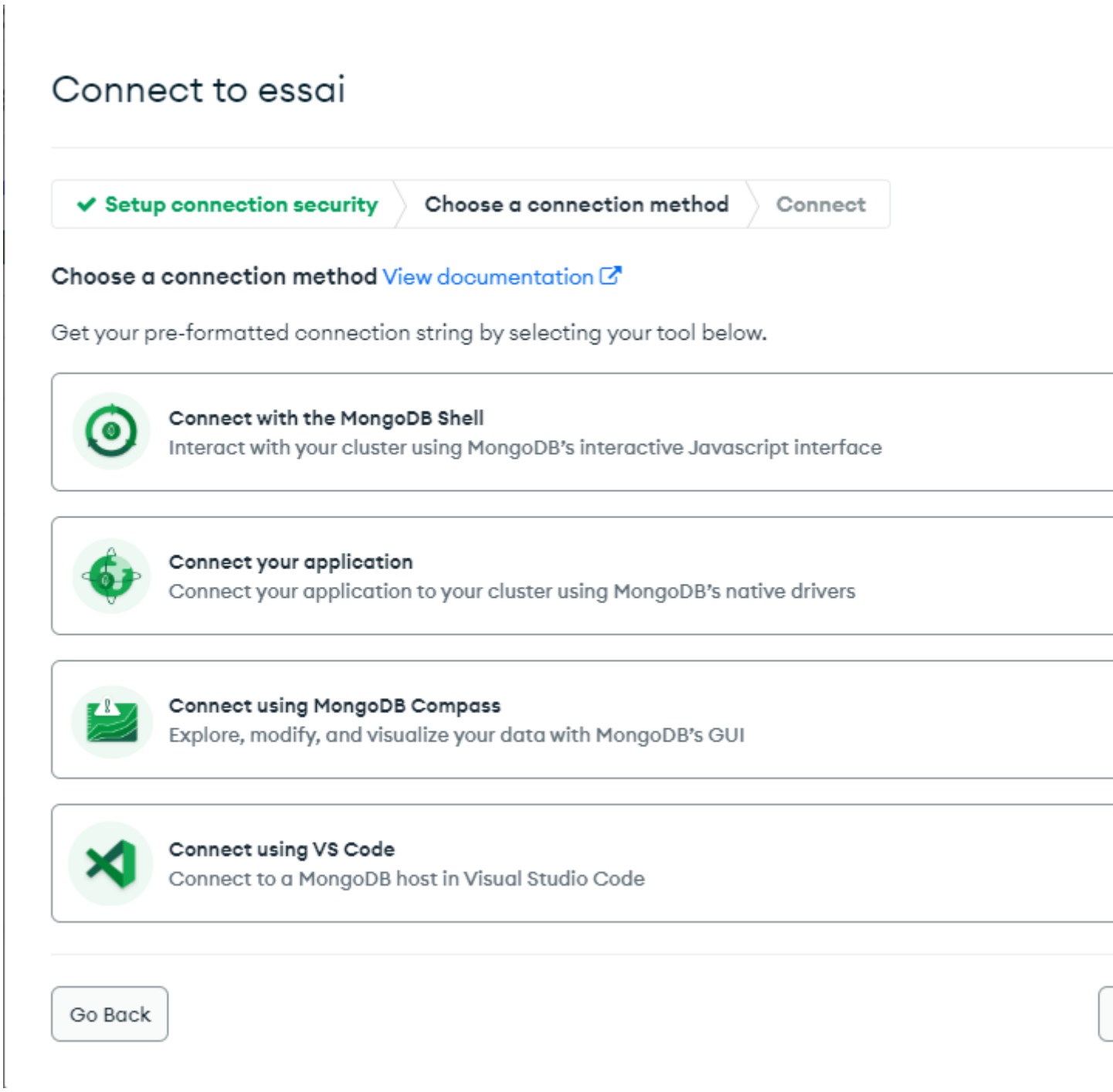
Dans cette partie, on crée un cluster shared nommé **essai** (c'est gratuit) :



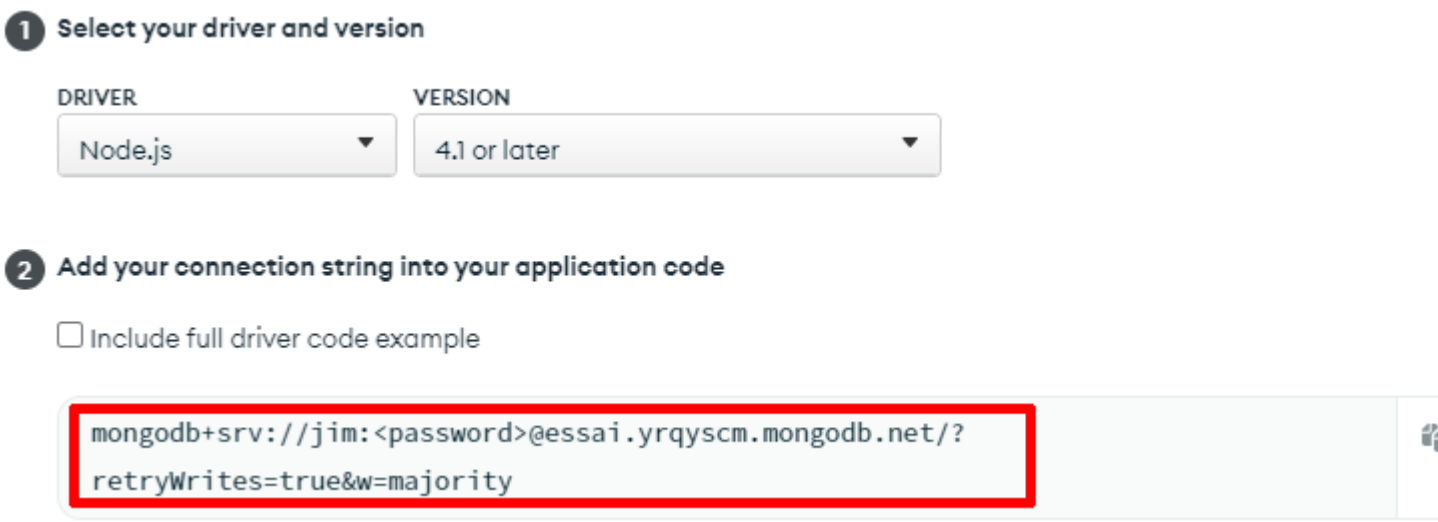
Cliquez sur le bouton **connect** :



Choisissez **Connect your application** :



Vous trouverez votre **chaîne de connexion**. Ajoutez le mot de passe de l'admin dans la chaîne à la place de <password>. Vous trouverez l'utilisateur admin en cliquant sur le dock de gauche sur Database Access. Ajoutez l'IP de votre machine dans la liste des IP autorisée sous l'onglet Network Access.



Gérez des variables d'environnement

Il est toujours bon de pouvoir différencier notre environnement de production de notre environnement de développement. Généralement, nous disposons d'une configuration nous permettant d'afficher les erreurs lorsque nous sommes en mode de développement, ce ne sera pas le cas lorsque nous serons en mode de production, dans ce cas, nous souhaiterons les logger sur un service externe ou dans un fichier par exemple.

Pour gérer la configuration de ces différents environnements de travail, nous aurons besoin d'un nouveau package, **env-cmd**, qui est une alternative au package **dotenv**.

Vous pouvez l'installer en entrant la commande suivante:
npm install env-cmd --save-dev

Créez à la racine de votre projet un dossier **env**. Puis à l'intérieur, créez les fichiers **.env**, **.env.dev** et **.env.prod**.

Maintenant que vous disposez de trois environnements distincts, vous voudrez sans doute pouvoir choisir l'environnement à lancer au démarrage de l'application. Pour permettre ce choix, vous

devrez modifier le script “**start**”, et créer deux nouveaux scripts vous permettant de lancer le mode de production et le mode de développement au besoin.
Ouvrez le fichier package.json:

- Modifiez le script “**start**” comme indiqué sur la capture pour utiliser le package env-cmd ainsi que le fichier **.env** général.
- Ajoutez le script “**dev**” et attachez-y la configuration pour lancer l’application en mode de développement avec le fichier **.env.dev**
- Ajoutez un dernier script, “**prod**” et répétez le dernier point mais pour le fichier **.env.prod**

Vous devriez obtenir le même résultat que sur cette capture :

```
package.json > ...
1   {
2     "name": "api",
3     "version": "0.0.0",
4     "private": true,
      ▷ Déboguer
5     "scripts": {
6       "start": "env-cmd -f ./env/.env nodemon ./bin/www",
7       "dev": "env-cmd -f ./env/.env.dev nodemon ./bin/www",
8       "prod": "env-cmd -f ./env/.env.prod nodemon ./bin/www"
9     },
10    "dependencies": {
11      "cookie-parser": "~1.4.4",
12      "debug": "~2.6.9",
13      "env-cmd": "^10.1.0",
14      "express": "~4.16.1",
15      "morgan": "~1.9.1",
16      "nodemon": "^2.0.20"
17    }
18  }
```

La connexion dans app.js

Nous allons maintenant nous concentrer sur l’installation et le paramétrage de tout ce qu’il vous faudra de manière à communiquer avec le SGBD MongoDB.
Installez le module **mongoose** pour gérer mongoDb et **cors** pour gérer le Cross Origin.
npm install mongoose cors --save

Maintenant que le package capable de prendre en charge la communication avec MongoDB est installé, nous allons créé une connexion entre la base de données distante, et notre projet.
Créez un répertoire **db** et à l’intérieur un fichier **mongo.js** dont voici le contenu (vous pouvez copier l’intégralité du code, attention toutefois à bien fournir la valeur de URL_MONGO dans le fichier .env précédemment créé) :

```
mongo.js ×   .env
db > mongo.js > ...
1   const mongoose = require('mongoose');
2
3   const clientOptions = {
4       useNewUrlParser    : true,
5       dbName             : 'apinode'
6   };
7
8   exports.initClientDbConnection = async () => {
9       try {
10          /* ATTENTION
11          On essaie de se connecté à mongoDB en utilisant la variable d'environneme
12          Il faut donc ne pas oublier de l'ajouter au fichier .env
13          URL_MONGO prends pour valeur la chaîne de connexion de votre cluster mong
14          */
15          await mongoose.connect(process.env.URL_MONGO, clientOptions)
16          console.log('Connected');
17      } catch (error) {
18          console.log(error);
19          throw error;
20      }
21  }
22
```

La méthode mongoose.connect() utilise la variable d'environnement URL_MONGO. N'oubliez pas de l'ajouter à vos fichiers .env.

```
NODE_ENV=template
APP_NAME=Mon API
API_URL=127.0.0.1
URL_MONGO = "mongodb+srv://jim:<password>@essai.******.mongodb.net/?retryWrites=true&w=ma
```

Modification de app.js

Ici, vous allez initialiser la connexion à mongoDB et utiliser le package **cors** qui va vous permettre d'autoriser les autres domaines et sous-domaines à envoyer des requêtes à votre API. En fin de fichier, déclarez votre route principale et un middleware pour gérer l'erreur 404.
Dans notre fichier, le '*' signifie que nous souhaitons accepter toutes les requêtes entrantes de tous les domaines externes ou non.
Vous noterez la présence de "cookie parser" et de "morgan" qu'il ne faudra pas oublier d'installer si ce n'est pas déjà fait.

app.js > ...

```javascript
/*
Le app.js reste le plus simple possible.
On initialise ici la connexion à MongoDB,
On utilise le package CORS installé précédemment en acceptant toutes les origines e
(pour récupérer le token d'authentification côté client)
On déclare notre route principale avec pour url de base "/".
On ajoute un retour en cas de requête sur une route inexistante (404).
*/

const express      = require('express');
const cookieParser = require('cookie-parser');
const logger       = require('morgan');
const cors         = require('cors');

const indexRouter = require('./routes/index');
const mongodb     = require('./db/mongo');

mongodb.initClientDbConnection();

const app = express();

app.use(cors({
    exposedHeaders: ['Authorization'],
    origin: '*'
}));
app.use(logger('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(cookieParser());

app.use('/', indexRouter);

app.use(function(req, res, next) {
    res.status(404).json({name: 'API', version: '1.0', status: 404, message: 'not_f
});

module.exports = app;
```

a

Déclarez des routes dans index.js et users.js

Pour notre projet, nous nous baserons sur un CRUD complet concernant des utilisateurs via une API. Libre à vous par la suite d'ajouter d'autres entités comme des livres, et j'en passe, le principe reste identique.

Dans le fichier **index.js**, la route **/users** doit charger un fichier contenant les routes propres aux users.

```
index.js
routes > index.js > ...
1   var express = require('express');
2   var router = express.Router();
3
4   const userRoute = require('../routes/users');
5
6   router.get('/', async (req, res) => {
7       res.status(200).json({
8           name    : process.env.APP_NAME,
9           version: '1.0',
10          status : 200,
11          message: 'Bienvenue sur l\'API !'
12      });
13  });
14
15  router.use('/users', userRoute);
16
17  module.exports = router;
18
19
```

Déclarez maintenant vos routes et les callbacks qui doivent s'exécuter. Vous pourriez faire de même pour tout type d'entité à exposer sur le web, des recettes de cuisines, des films, etc.
Créez donc un dossier 'routes' si ce n'est pas déjà fait, et ajoutez le fichier users.js. Copiez le contenu suivant :

users.js

routes > users.js > ...

```
/*
On utilise le routeur d'express pour définir 4 routes.
On exprime ici ce qu'on appelle un CRUD pou Create, Read, Update, Delete
Pour déclarer une route dans express : app.verbeHttp(route, fonction)
Ici on utilise le router pour appeler le verbe Http
router.get('/:id', service.getById);
le callback sera fourni par un service qu'on va déclarer
Recourir à des services permet de structurer le projet et séparer les diverses lo
On pourra se resservir de ces mêmes services pour d'autres entités que users
*/
const express = require('express');
const router = express.Router();

const service = require('../services/users');
// La route pour lire les infos d'un utilisateur
router.get('/:id', service.getById);
// La route pour ajouter un utilisateur
router.put('/add', service.add);
// La route pour modifier un utilisateur
router.patch('/:id', service.update);
// La route pour supprimer un utilisateur
router.delete('/:id', service.delete);

module.exports = router;
```

Définissez le modèle de données

Vous y êtes presque, maintenant que nous avons nos routes et mis en place toute notre architecture, il ne nous reste qu'à définir notre modèle "users", de cette manière, nous pourrons faire persister les données de nos users dans le SGBD MongoDB.
Créez un répertoire **models** dans lequel vous déclarerez un fichier **user.js** qui servira de modèle de donnée à MongoDB.
Nos utilisateurs disposeront de mots de passe, nous devrons donc faire en sorte d'enregistrer ces mots de passe de manière chiffrée dans la base de données, c'est la raison pour laquelle vous allez installer le module **bcrypt** :

Maintenant que nous sommes prêts, nous allons pouvoir créer différents services qui vont nous permettre de manipuler les données des entités User présentes dans notre base de données.

npm install bcrypt --save

Ensuite, il ne nous reste qu’a ajouter le code de notre entité comme suit :

user.js

models > user.js > ...

```javascript
const mongoose = require('mongoose');
const Schema   = mongoose.Schema;
// On importe le module bcrypt qui permet de hacher des expressions
const bcrypt   = require('bcrypt');

const User = new Schema({
    name: {
        type    : String,
        trim    : true,
        required: [true, 'Le nom est requis']
    },
    firstname: {
        type: String,
        trim: true
    },
    email: {
        type     : String,
        trim     : true,
        required : [true, 'L’email est requis'],
        unique   : true, // index unique
        lowercase: true
    },
    password: {
        type: String,
        trim: true,
    }
}, {
    // ajoute 2 champs au document createdAt et updatedAt
    timestamps: true
});

// Hash le mot de passe quand il est modifié
User.pre('save', function(next) {
    if (!this.isModified('password')) {
        return next();
    }

    this.password = bcrypt.hashSync(this.password, 10);

    next();
});

module.exports = mongoose.model('User', User);
```

Le service

Créez un répertoire **services** et, à l'intérieur, un fichier **users.js**.
La méthode suivante vous permettra de récupérer un utilisateur en ayant fourni un ID. Le procédé pour récupérer une entité en base de données avec MongoDB est toujours sensiblement le même, vous pourriez donc de la même manière créer une méthode permettant de récupérer un utilisateur par son email par exemple.

```
users.js
services > users.js > getById > getById
1
2   // On importe le modèle de données
3   const User = require('../models/user');
4
5
6   //On exporte le callback afin d'y accéder dans notre gestionnaire de routes
7   //Ici c'est le callback qui servira à ajouter un user avec son id
8   exports.getById = async (req, res, next) => {
9       const id = req.params.id
10
11      try {
12          let user = await User.findById(id);
13
14          if (user) {
15              return res.status(200).json(user);
16          }
17
18          return res.status(404).json('user_not_found');
19      } catch (error) {
20          return res.status(501).json(error);
21      }
22  }
23
```

Cette méthode quant à elle, permettra l'ajout d'un utilisateur en base de données. Attention à la règle d'or que je n'ai pas appliqué ici, à savoir "Never Trust User Inputs", j'aurais donc dû vérifier les différentes données fournies **avant** l'enregistrement en base de données, n'oubliez jamais de le faire !

Dans la même idée, vous ajouterez également une méthode qui permettra de mettre à jour un utilisateur existant en base de données, ainsi que la méthode permettant les suppressions.

Encore une fois, ne pas oublier de vérifier les données avant toute insertion en base de données.

```
//Ici c'est le callback qui servira à ajouter un user
exports.add = async (req, res, next) => {

    const temp = ({
        name     : req.body.name,
        firstname: req.body.firstname,
        email    : req.body.email,
        password : req.body.password
    });


    try {
        let user = await User.create(temp);

        return res.status(201).json(user);
    } catch (error) {
        return res.status(501).json(error);
    }
}
```

```
//Ici c'est le callback qui servira à modifier un user
exports.update = async (req, res, next) => {
    const id = req.params.id
    const temp = ({
        name     : req.body.name,
        firstname: req.body.firstname,
        email    : req.body.email,
        password : req.body.password
    });

    try {
        let user = await User.findOne({_id : id});

        if (user) {
            Object.keys(temp).forEach((key) => {
                if (!!temp[key]) {
                    user[key] = temp[key];
                }
            });

            await user.save();
            return res.status(201).json(user);
        }

        return res.status(404).json('user_not_found');
    } catch (error) {
        return res.status(501).json(error);
    }
}
//Ici c'est le callback qui servira à supprimer un user
exports.delete = async (req, res, next) => {
    const id = req.params.id

    try {
        await User.deleteOne({ _id: id });

        return res.status(204).json('delete_ok');
    } catch (error) {
        return res.status(501).json(error);
    }
}
```

Démarrer l'API

Maintenant que tout notre projet est prêt, il ne nous reste qu'a le démarrer de manière à pouvoir tout tester. Tapez **npm run start** ou **dev** et, si tout est bien configuré, vous verrez :

```
PS C:\Users\jeanm\Desktop\api> npm start

> api@0.0.0 start
> env-cmd -f ./env/.env nodemon ./bin/www

[nodemon] 2.0.20
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node ./bin/www`
mongoDb Connected
```

Tester l'API

Pour tester l'API et les requêtes, utilisez**Postman**.
Installez-le et démarrez-le. Si vous possédez déjà un compte alors utilisez le, sinon, créez en un depuis leur site ou directement depuis l'application.

Ajoutez un utilisateur en envoyant une requête **PUT** (on utilise généralement une requête POST pour l'ajout d'une tout nouvelle entité) sur l'URL **localhost :3000/users/add** en passant les paramètres nécessaires dans le body au format x-www-form-urlencoded (voir image ci-dessous).

PUT localhost:3000/users/a

localhost:3000/users/add

PUT localhost:3000/users/add

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL

KEY	VALUE
name	doe
firstname	john
email	john@xxx.com
password	1234
Key	Value

Body Cookies Headers (9) Test Results

Pretty Raw Preview Visualize JSON


```
{
    "name": "doe",
    "firstname": "john",
    "email": "john@xxx.com",
    "password": "$2b$10$Fxjr9lMzQoxHJgtJssDLbeigJsq4BEVBD0IX2T.ueDUgKPrWz8gAy",
    "_id": "63691e1fa40813418e08c842",
    "createdAt": "2022-11-07T15:02:55.938Z",
    "updatedAt": "2022-11-07T15:02:55.938Z",
    "__v": 0
}
```


Vérifiez côté mongoDB si l'objet est bien en base. Copiez-collez l'**id** ainsi créé pour tester votre route en GET avec l'URL **localhost :3000/users/**id user**** (voir ci-dessous).

GET localhost:3000/users/6

localhost:3000/users/6369296a9f7060a227f8e237

GET localhost:3000/users/6369296a9f7060a227f8e237

Params Authorization Headers (7) Body Pre-request Script Tests Settings

Query Params

KEY	VALUE
Key	Value

Body Cookies Headers (9) Test Results

Pretty Raw Preview Visualize JSON


```
{
    "_id": "6369296a9f7060a227f8e237",
    "name": "doe",
    "firstname": "john",
    "email": "john@xxx.com",
    "password": "$2b$10$SbsaaEijEP1.XSn8wGzIZeuDAPTAVTUKdgPLXUwVHZj5UhRRN5oXy",
    "createdAt": "2022-11-07T15:51:06.301Z",
    "updatedAt": "2022-11-07T15:51:06.301Z",
    "__v": 0
}
```


Changez le mot de passe en testant la route en **PATCH** pour modifier le user (voir ci-dessous).

PATCH localhost:3000/users

localhost:3000/users/6369296a9f7060a227f8e237

PATCH localhost:3000/users/6369296a9f7060a227f8e237

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL

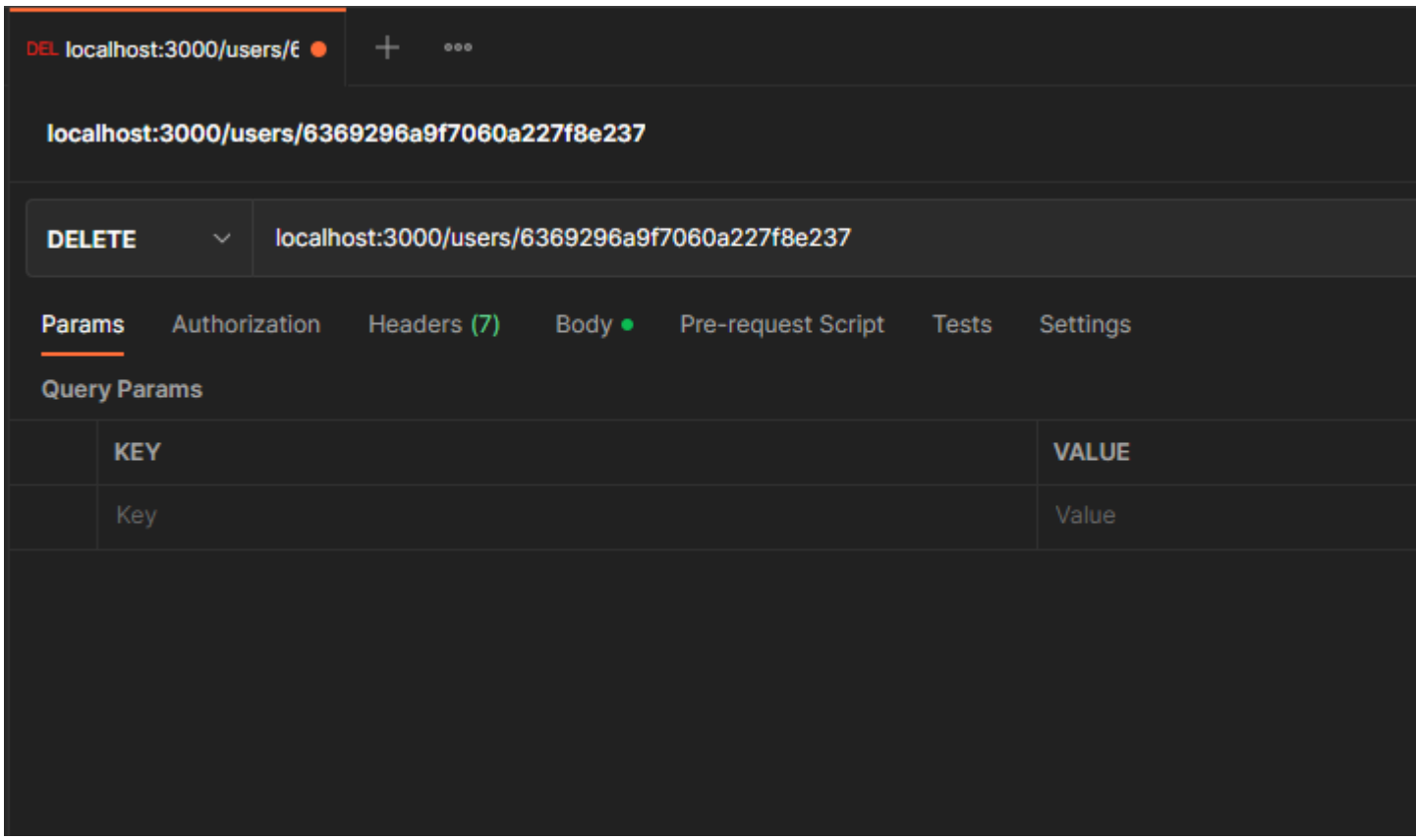
KEY	VALUE
name	doe
firstname	john
email	john@xxx.com
password	1236666
Key	Value

Body Cookies Headers (9) Test Results

Pretty Raw Preview Visualize JSON


```
{
    "_id": "6369296a9f7060a227f8e237",
    "name": "doe",
    "firstname": "john",
    "email": "john@xxx.com",
    "password": "$2b$10$pDuMfbWE7tCOH6F10MOiredflLsL8n2n/q.NEit1iwBSgg5y50txy",
    "createdAt": "2022-11-07T15:51:06.301Z",
    "updatedAt": "2022-11-07T15:57:56.931Z",
    "__v": 0
}
```


Enfin, supprimez l'utilisateur grâce à son id en utilisant **DELETE** et vérifiez qu'il a bien été supprimé côté mongoDB.



a

Pour aller plus loin : MongoDB Compass

Téléchargez et utilisez l'interface mongoDB Compass.

Je retiens : l'API RESTful

Pour créer une API, il faut schématiser la donnée en utilisant l'objet Schema de mongoose, un service pour les opérations en base et les callbacks pour chaque route.

POST et PUT sont deux verbes HTTP pour créer la donnée, le verbe GET vous sert à accéder à la donnée et à la lire, les verbes PATCH ou PUT servent à mettre à jour et DELETE à supprimer.

Modifiez l'API avec JSON Web Token (JWT)

Nous allons modifier l'API réalisée dans le tutoriel précédent en créant un système basique d'authentification et de privatisation de routes.
Pour sécuriser les routes on va s'appuyer sur JWT, une méthode standard pour sécuriser l'échange de données entre deux tiers ou plusieurs.

Fonctionnement

- Le client se connecte en envoyant un mot de passe au serveur.
- Le serveur vérifie que les informations passées sont valides.
- Si les informations sont validées, il génère un token (jeton) signé.

Le token

Il est composé de trois objets :

- Un header contenant la description du **token.**
- Les informations que l'on passe ou **payload.**
- Une **signature** combinant une **clé secrète avec le payload et la description du token**.

Côté client, ce token sera récupéré et stocké en cookie ou variable de session, par exemple. Le client pourra alors refaire des requêtes en utilisant ce token.

Installation

Après ce petit morceau de théorie rapide, allons-y pour la pratique, vous allez mettre en place votre premier système d'authentification avec un token JWT.

Premièrement, installez le module JWT avec la commande :
npm install jsonwebtoken –save.

Dans le fichier **.env**, déclarez la clé secrète : **SECRET_KEY= GTGh6rdP54GT76**

```
NODE_ENV=template
APP_NAME=Mon API
API_URL=127.0.0.1
URL_MONGO = "mongodb+srv://jim:tJGb47y65jVxHTjW@essai.yrqyscm.mongodb.net/?retryWrites=tr
SECRET_KEY= GTGh6rdP54GT76
```

C'est le développeur qui définit la clé, vous auriez pu utiliser n'importe quoi d'autre en tant que clé, et même utiliser un service externe sur le Web pour vous en créer un.

Modifiez le service

Dans le fichier **services/user.js**, ajoutez les imports de **JWT** et de **bcrypt**. Bcrypt servira à comparer le password à celui qui est enregistré dans mongoDB.

```
users.js services ×    users.js routes
services > users.js > ...
1   const User = require('../models/user');
2   const bcrypt = require('bcrypt');
3   const jwt    = require('jsonwebtoken');
4
```

Dans le même fichier, déclarez également la méthode **authenticate** qui se chargera de la vérification du mot de passe.

```
exports.authenticate = async (req, res, next) => {
    const { email, password } = req.body;

    try {
        let user = await User.findOne({ email: email }, '-__v -createdAt -updatedAt');

        if (user) {
            bcrypt.compare(password, user.password, function(err, response) {
                if (err) {
                    throw new Error(err);
                }
                if (response) {
                    delete user._doc.password;

                    const expireIn = 24 * 60 * 60;
                    const token    = jwt.sign({
                        user: user
                    },
                    SECRET_KEY,
                    {
                        expiresIn: expireIn
                    });

                    res.header('Authorization', 'Bearer ' + token);

                    return res.status(200).json('authenticate_succeed');
                }

                return res.status(403).json('wrong_credentials');
            });
        } else {
            return res.status(404).json('user_not_found');
        }
    } catch (error) {
        return res.status(501).json(error);
    }
}
```

Dans le code juste en haut :

- On récupère l'adresse email ainsi que le mot de passe en clair envoyé dans le body de la requête.
- On récupère l'utilisateur en base de données en faisant une recherche à l'aide de l'adresse email qui a été fournie (récupérée juste en haut)

- On utilise la librairie **bcrypt** pour vérifier que le mot de passe en clair correspond au mot de passe chiffré qui est enregistré en base de données pour cet utilisateur.
- Dans la fonction callback, on teste si une erreur a été levée, si pas, on génère le token pour l'envoyer dans le header de la réponse, qui elle, contient un code 200 (ok) avec un simple message.

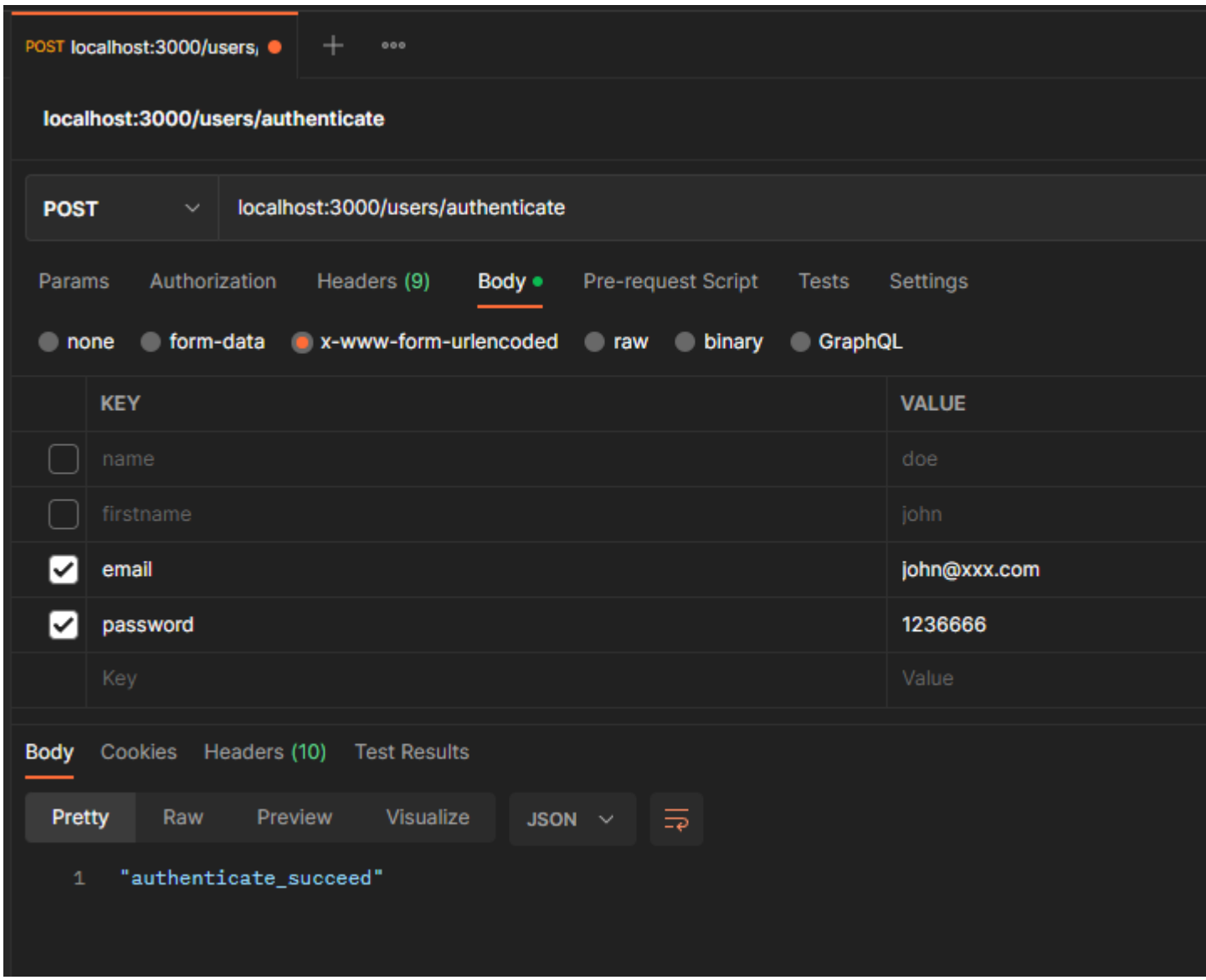
Ajoutez une nouvelle route

Maintenant qu'il est possible pour un utilisateur d'obtenir un token au moment de son authentification, il vous faut mettre en place la route **/authenticate** dans le fichier **routes/users.js.**

```
users.js services    users.js routes ×
routes > users.js > ...
1
2   const express = require('express');
3   const router = express.Router();
4
5   const service = require('../services/users');
6   router.get('/:id', service.getById);
7   router.put('/add', service.add);
8   router.patch('/:id', service.update);
9   router.delete('/:id', service.delete);
10
11  // Ajout de la route /authenticate
12  router.post('/authenticate', service.authenticate);
13
14  module.exports = router;
15
```

Testez la connexion d'un utilisateur

Démarrez l'API avec **npm start** et testez une requête en **POST** avec l'URL **localhost :3000/users/authenticate** en envoyant l'**e-mail** et le **password** comme s'ils provenaient d'un formulaire (x-www-form-urlencoded comme sur l'image suivante).



Sous l'onglet Headers de la réponse, vous trouverez le **token** sous **Authorization**.

Body Cookies **Headers** (10) Test Results

KEY
X-Powered-By
Access-Control-Allow-Origin
Access-Control-Expose-Headers
Authorization
Content-Type
Content-Length
ETag
Date
Connection
Keep-Alive

Privatisez des routes

Vous allez déclarer un **intergiciel (middleware)** qui sera exécuté **avant** les appels à **une fonction de service** et se chargera de vérifier la **validité du token pour laisser passer ou non la requête du client**.
Pour cela, créez un répertoire **middlewares** et, à l'intérieur, un fichier **private.js**.

private.js

middlewares > private.js > checkJWT > checkJWT

```javascript
const jwt        = require('jsonwebtoken');
const SECRET_KEY = process.env.SECRET_KEY;

exports.checkJWT = async (req, res, next) => {
    let token = req.headers['x-access-token'] || req.headers['authorization'];
    if (!!token && token.startsWith('Bearer ')) {
        token = token.slice(7, token.length);
    }

    if (token) {
        jwt.verify(token, SECRET_KEY, (err, decoded) => {
            if (err) {
                return res.status(401).json('token_not_valid');
            } else {
                req.decoded = decoded;

                const expiresIn = 24 * 60 * 60;
                const newToken  = jwt.sign({
                    user : decoded.user
                },
                SECRET_KEY,
                {
                    expiresIn: expiresIn
                });

                res.header('Authorization', 'Bearer ' + newToken);
                next();
            }
        });
    } else {
        return res.status(401).json('token_required');
    }
}
```

Pour vous servir de votre middleware, importez-le dans le fichier **routes/users.js** et appelez la fonction **checkJWT()** sur les routes que vous souhaitez protéger.

users.js

routes > users.js > ...

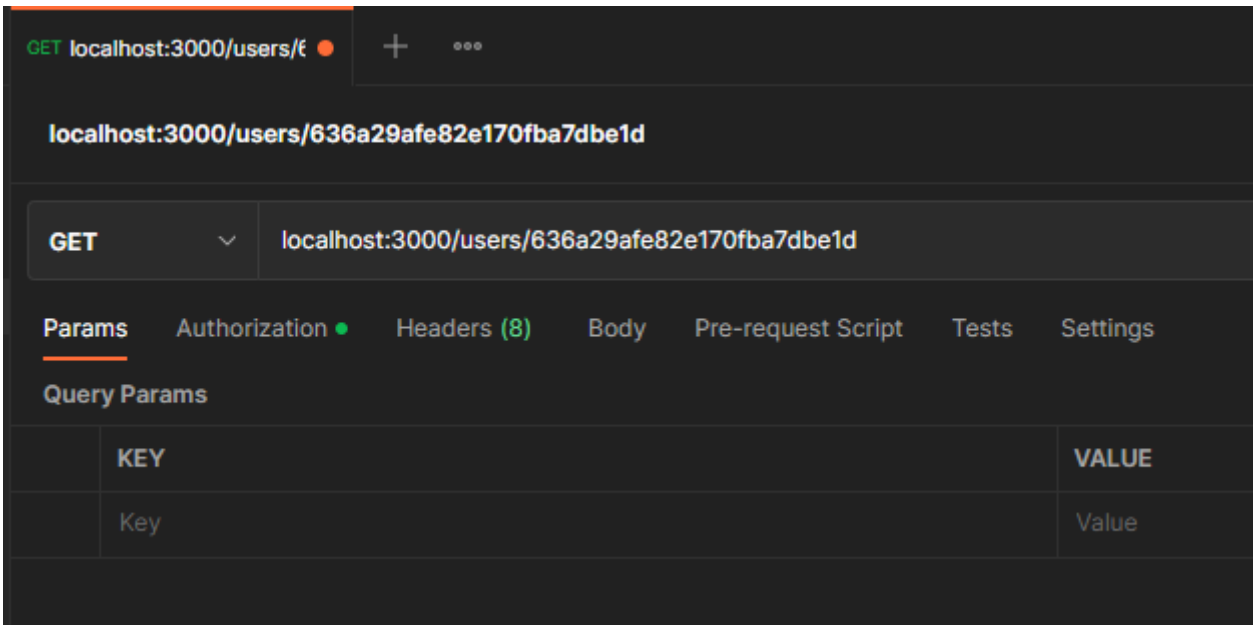
```
1
2   const express = require('express');
3   const router = express.Router();
4
5   const service = require('../services/users');
6
7   const private = require('../middlewares/private');
8
9   router.get('/:id', private.checkJWT, service.getById);
10
11  router.put('/add', service.add);
12
13  router.patch('/update', private.checkJWT, service.update);
14
15  router.delete('/delete', private.checkJWT, service.delete);
16
17  router.post('/authenticate', service.authenticate);
18
19  module.exports = router;
20
21
22
```

Tests d’accès à une route privée

Commencez par tester la route GET sans être considéré comme connecté à l’API, c’est-à-dire sans le token.

Test d’accès à la route GET sans le token

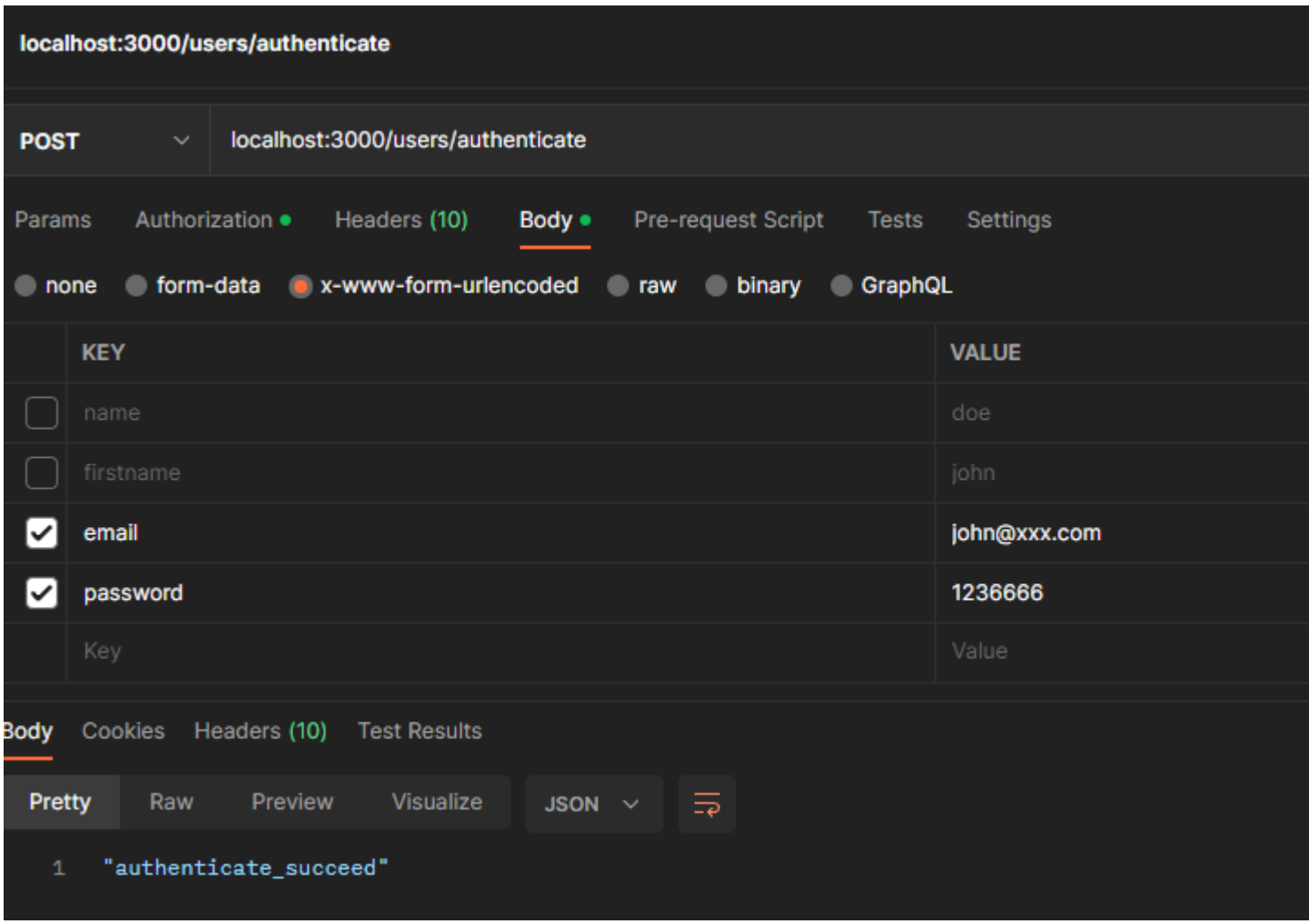
Envoyez une requête en GET en utilisant la route **localhost :3000/user/xxxxxxxx** où xxxxxxxx est l’id de l’utilisateur dans mongoDB.



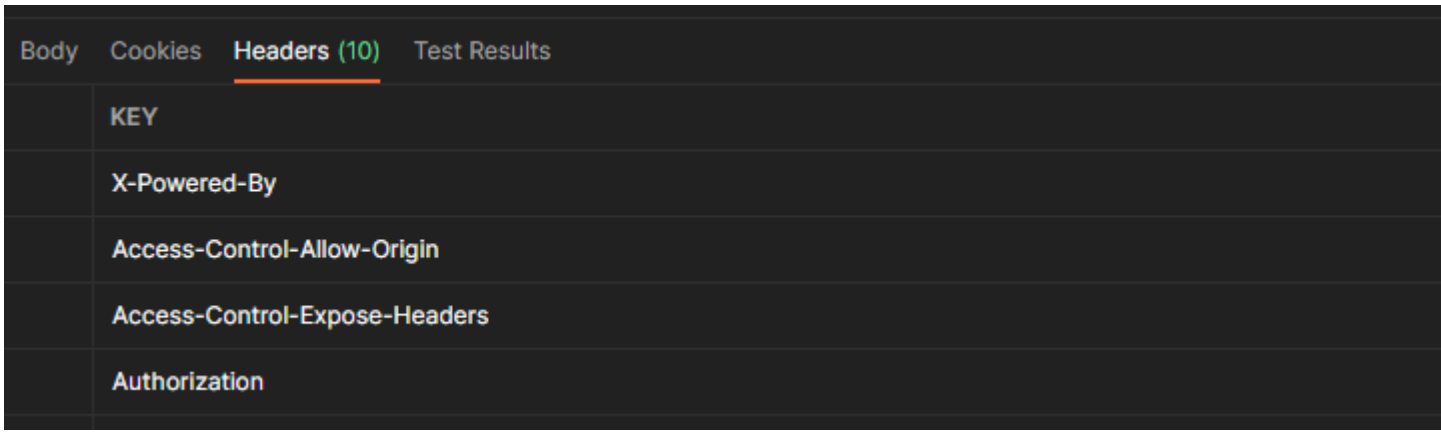
Vous devriez voir dans le corps de la réponse l'expression « **token required** » ce qui signifie que la route est bien sécurisée. Maintenant, vous allez connecter l'utilisateur et utiliser le token généré pour tester la même route.

Test d'accès à la route GET avec le token

Servez-vous de Postman pour connecter l'utilisateur que vous avez créé précédemment en utilisant l'**e-mail** et le **password** en paramètres (voir image ci-dessous).



Rendez-vous sous l'onglet **Headers** de la réponse pour copier le token.



Envoyez une nouvelle requête en GET en utilisant le **token** cette fois ci, comme le montre l'image ci-dessous.

POST localhost:3000/users, GET localhost:3000/users/6

localhost:3000/users/636a29afe82e170fba7dbe1d

GET localhost:3000/users/636a29afe82e170fba7dbe1d Id user

Params Authorization Headers (8) Body Pre-request Script Tests Settings

Type Bearer Token

Heads up! These parameters hold sensitive data. variables

The authorization header will be automatically generated when you send the request. Learn more about authorization

Sélectionner Bearer token

Token

Body Cookies Headers (10) Test Results

Pretty Raw Preview Visualize JSON


```
{
    "_id": "636a29afe82e170fba7dbe1d",
    "name": "doe",
    "firstname": "john",
    "email": "john@xxx.com",
    "password": "$2b$10$Io7gRziv3ohzd0mAku8WC.Xv.FSXXhCvXWLN2I3LCW3Oufgf8cwem",
    "createdAt": "2022-11-08T10:04:31.719Z",
    "updatedAt": "2022-11-08T10:04:31.719Z",
    "__v": 0
}
```


Dans le **body de la réponse**, vous allez voir apparaître votre objet utilisateur et ses informations comme sur l'image ci-dessous.

Body Cookies Headers (10) Test Results

Pretty Raw Preview Visualize JSON


```
{
    "_id": "636a29afe82e170fba7dbe1d",
    "name": "doe",
    "firstname": "john",
    "email": "john@xxx.com",
    "password": "$2b$10$Io7gRziv3ohzd0mAku8WC.Xv.FSXXhCvXWLN2I3LCW3Oufgf8cwem",
    "createdAt": "2022-11-08T10:04:31.719Z",
    "updatedAt": "2022-11-08T10:04:31.719Z",
    "__v": 0
}
```


Voilà, vous venez de mettre en place un système d'authentification simple et vous avez utilisé JWT pour privatiser les routes de votre API.

Créez d'autres données que va servir votre API, comme une liste de livres par exemple.

Gérez les fichiers entrants

Pour gérer des fichiers utilisateurs, vous utiliserez le module **multer**. Il permet d'uploader les fichiers dans un répertoire de l'application. Vous créerez ensuite un **CRUD** pour ajouter, lire, modifier et supprimer ces fichiers.

Installation de multer

Installez multer en entrant la commande : **npm install multer, et attendez la fin de l'installation.**

Lieu de sauvegarde des fichiers entrants

Avant de pouvoir commencer à uploader des fichiers, vous allez devoir créer un répertoire **uploads** à la racine de l'API que vous avez créée et sécurisée dans les deux cours précédents.
C'est dans ce répertoire que vous allez sauvegarder les fichiers que l'utilisateur va téléverser à votre API.
Si vous souhaitez utiliser un autre nom que celui choisi, libre à vous de le faire, cela ne changera en rien la manière de fonctionner, mais il vous faudra spécifier ce nom en tant que répertoire cible lorsque l'utilisateur téléversera les fichiers.

Le middleware multer

Dans notre exemple, le middleware multer se chargera de l'**upload** (téléversement) du fichier et sera appelé juste **après l'authentification**. Le principe reste le même si vous souhaitez dédier une route à un téléversement en particulier.
Créez un fichier **files-storage.js** dans le répertoire **middlewares**. **Multer** dispose d'une méthode **diskStorage()** pour enregistrer des fichiers sur votre disque.
Elle prend deux arguments :
destination : une fonction callback qui va vous permettre de déclarer le répertoire de destination.
filename : une fonction callback qui va vous permettre de renommer le fichier avant de le sauvegarder pour éviter que des fichiers portent le même nom. Vous remplacerez les espaces dans les noms des fichiers par des underscore en utilisant la méthodes **split()** pour découper la chaîne de caractères puis **join()** pour la recomposer avec des underscores, car ces espaces dans les noms de fichiers peuvent causer des problèmes côté serveur.

Déclarez un objet **MIME_TYPES** avec les extensions de fichiers que votre API accepte.

files-storage.js ×

middlewares > files-storage.js > ...

```
1   const multer = require('multer');
2
3   const MIME_TYPES = {
4       'image/jpg': 'jpg',
5       'image/jpeg': 'jpg',
6       'image/png': 'png'
7   };
8
9   const storage = multer.diskStorage({
10      destination: (req, file, callback) => {
11          callback(null, 'uploads');
12      },
13      filename: (req, file, callback) => {
14          const name = file.originalname.split(' ').join('_');
15          console.log(name)
16          const extension = MIME_TYPES[file.mimetype];
17          // Ci-dessous le callback ajoute un timestamp concaténé avec
18          //le nom de l'image comme nom d'image pour la sauvegarde
19          // Cela permet de la rendre unique.
20          callback(null, Date.now() + name);
21      }
22  });
23
24  module.exports = multer({storage: storage});
25
```

Création d’un modèle pour les fichiers

Créez un fichier **file.js** dans le répertoire **models**. Ajoutez le schéma suivant dans le fichier :

```
file.js
models > file.js > ...
1   const mongoose = require('mongoose');
2
3   const fileSchema = mongoose.Schema({
4       name: { type: String, required: true },
5       description: { type: String, required: true },
6       fileUrl: { type: String, required: true },
7       userId: { type: String, required: true },
8   });
9
10  module.exports = mongoose.model('File', fileSchema);
```

Ce sont les champs requis pour enregistrer un fichier uploadé sur mongoDB.

Création des routes pour gérer les fichiers

Ajoutez un fichier **files.js** dans le répertoire **routes**. Pour les routes en **POST** et en **PUT**, on utilise le middleware **multer** pour la sauvegarde des fichiers. On le fait intervenir **après l’authentification**, c’est important car on souhaite que tout upload ne puisse être possible qu’après authentification.

index.js

routes > index.js > ...

```javascript
var express = require('express');
var router = express.Router();

const userRoute = require('../routes/users');
const fileRoute = require('../routes/files');

router.get('/', async (req, res) => {
    res.status(200).json({
        name    : process.env.APP_NAME,
        version: '1.0',
        status : 200,
        message: 'Bienvenue sur l\'API !'
    });
});

router.use('/users', userRoute);
router.use('/files', fileRoute);

module.exports = router;
```

Prise en charge du répertoire files

L’application a besoin de savoir où sauvegarder les fichiers. Modifiez **app.js** :

files.js routes

routes > files.js > ...

```
1   const express = require('express');
2
3   const router = express.Router();
4
5   const service = require('../services/files');
6
7   const multer = require('../middlewares/files-storage');
8
9   const private = require('../middlewares/private');
10
11
12  router.get('/', private.checkJWT, service.getAllFiles);
13
14  router.post('/', private.checkJWT, multer, service.createOneFile);
15
16  router.get('/:id', private.checkJWT, service.getOneFile);
17
18  router.put('/:id', private.checkJWT, multer, service.modifyOneFile);
19
20  router.delete('/delete', private.checkJWT, service.deleteOneFile);
21
22  module.exports = router;
23
24
```

Sauvegarder une image

Ajoutez un fichier **files.js** dans le répertoire **services**. Le premier callback à déclarer est **createOneFile** qui récupère les informations dans la requête pour instancier un objet File (avec l'id de l'utilisateur pour savoir à qui appartient l'image) et le sauvegarder dans mongoDB.

```
files.js
services > files.js > ...
1   const { storage } = require('../middlewares/files-storage');
2   const File = require('../models/file');
3   const fs = require('fs')
4
5   //Méthode chargée de créer un fichier
6   exports.createOneFile = (req, res, next) => {
7       const file = new File({
8           name: req.file.filename,
9           imageUrl: `${req.protocol}://localhost:3000/uploads/${req.file.filename}`
10          userId: req.body.userId
11      });
12
13      file.save()
14      .then(() => { res.status(201).json({message: 'Objet enregistré !'})})
15      .catch(error => { res.status(400).json( { error })})
16  };
17
```

Pour tester, créez un formulaire html en ajoutant une vue à votre api, qui sera chargée d'envoyer le fichier et les informations pour la sauvegarde en base. Pour cela, Node.Js dispose d'un moteur de vue qui est ejs.

Installez-le **: npm install ejs**. Informez l'application qu'elle va utiliser ce moteur de vue avec le répertoire **views** en ajoutant à **app.js** :

```
app.set('views', path.join(__dirname, 'views'))
app.set('view engine', 'ejs')
```

Créez un répertoire **views** et, à l'intérieur, un fichier **index.ejs** avec les deux formulaires permettant de se connecter et de tester l'envoi d'une image :

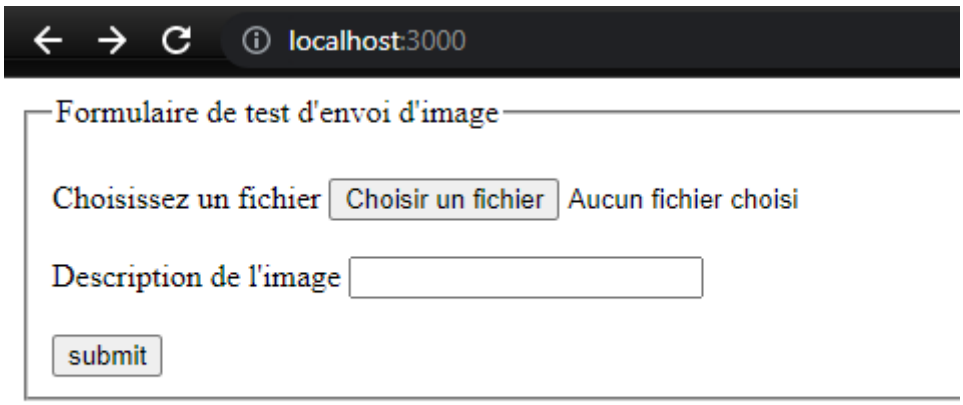
```
index.ejs
views > index.ejs > ...
1   <form method="post" action="http://localhost:3000/files" enctype="multipart/form-
2       <fieldset>
3           <legend>Formulaire de test d'envoi d'image</legend><br>
4           <label for="upload_file"> Choisissez un fichier</label>
5           <input type="file" name="upload_file"><br><br>
6           <label for="upload_file"> Description de l'image</label>
7           <input type="text" name="description"><br><br>
8
9           <input type="text" name="userId" value="636a29afe82e170fba7dbe1d" hidden>
10
11          <input type="submit" value="submit">
12      </fieldset>
13  </form>
```

Remarquez le champ souligné. Mettez ici **l'id de l'utilisateur** enregistré dans mongoDb. L'objectif ici n'est que tester rapidement que l'envoi et l'enregistrement du fichier est correct, vous pourrez retirer cette partie par la suite si vous le souhaitez, ou la conserver à titre d'exemple si vous souhaitez aller plus loin avec ce moteur de vues / moteur de templates.

La méthode **res.render()** permet de **rendre une vue** (fichier ejs). Modifiez le gestionnaire de route de la racine de votre API dans **index.js** :

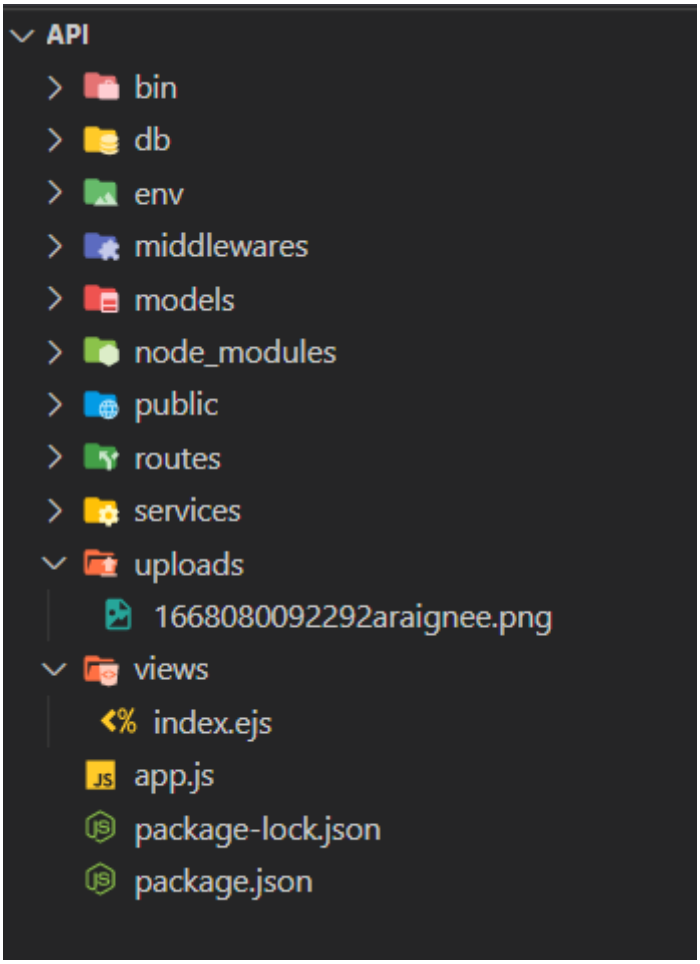
```
router.get('/', async (req, res) => {
/*      res.status(200).json({
            name    : process.env.APP_NAME,
            version: '1.0',
            status : 200,
            message: 'Bienvenue sur l\'API !'
        }); */

        res.render('index', {
            title: 'Accueil'
        })
});
```

Démarrez l'application et rendez-vous à la racine dans votre explorateur :

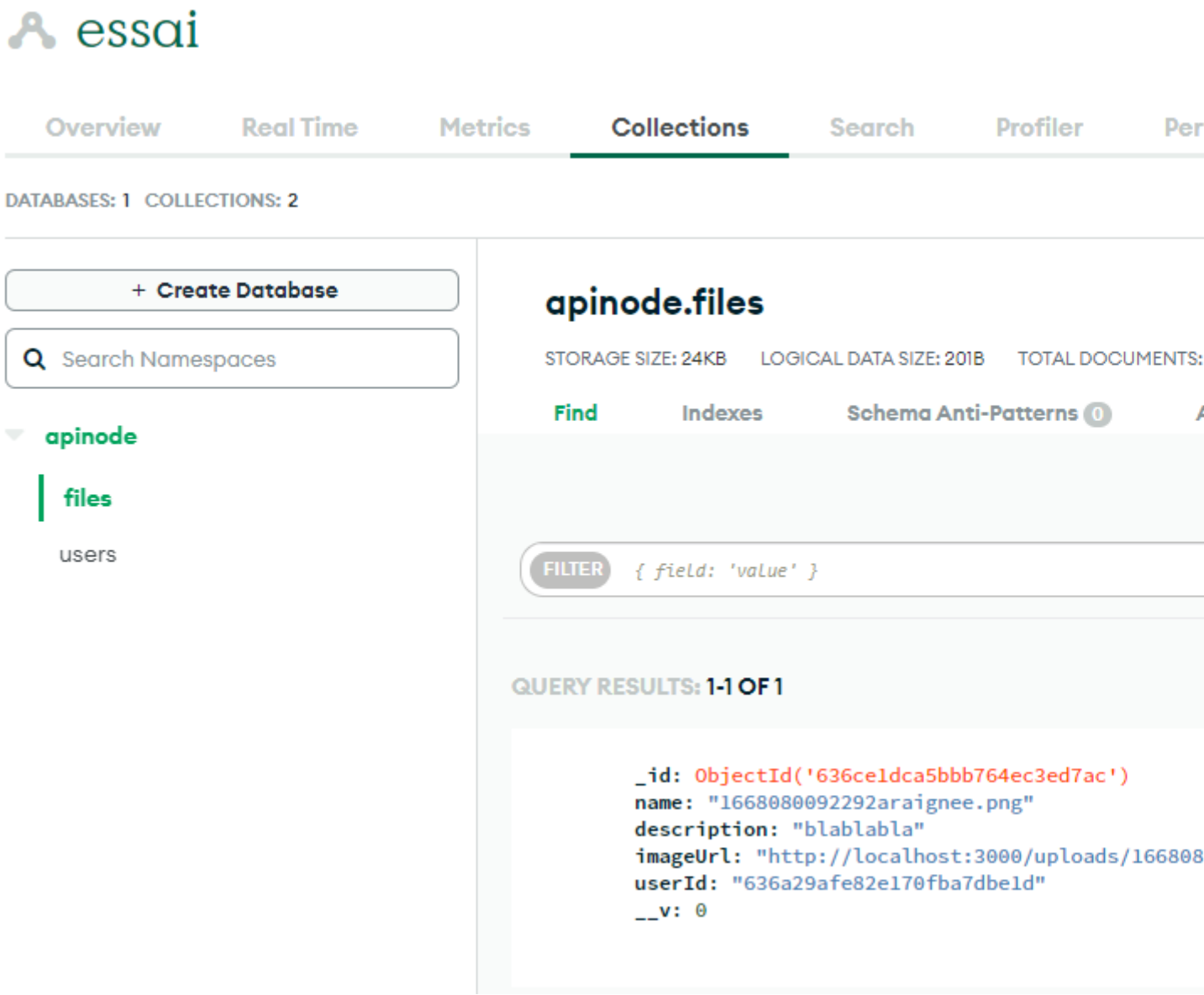


Pour faire le test sans avoir besoin de vous connectez, retirez le middleware **private.checkJWT** dans le gestionnaire de route **post()** de **routes/files.js**.

Un fois le fichier uploadé via cette vue de test, vous devriez voir apparaître votre image dans **uploads** avec le **timestamp** devant son nom :



Vérifiez si l'objet représentant cette image est présent côté mongoDB :



Charger une image

Nous avons enregistré les informations utiles de l'image uploadée directement dans la base de données. La récupérer sera donc un jeu d'enfant puisque nous avons enregistré son url. Attention que pour l'exemple, j'ai également enregistré l'hôte ainsi que le port (localhost:3000). Dans la réalité, nous n'enregistrerons que le chemin APRES l'hôte de manière à pouvoir avoir de bons chemins vers les images même dans le cas où l'on souhaite changer de serveur ou de technologie (passage à PHP ou Python par exemple). L'url "http://localhost:3000/uploads/mon_image.png" qui se trouve dans la base de données devrait être idéalement stockée de la manière suivante "mon_image.png", à vous ensuite de reconstituer le chemin vers l'image depuis le code source pour que l'url soit identique à celle fournie avec le nom de l'hôte.

Pour récupérer l'url de notre image, nous allons utiliser son id dans la méthode **findOne()** de mongoose afin de récupérer les données depuis la base de données.

```
//Méthode chargée de récupérer un fichier
exports.getOneFile = (req, res, next) => {
    File.findOne({ _id: req.params.id })
    .then(file => res.status(200).json(file))
    .catch(error => res.status(404).json({ error }));
};
```

Modifier une image

Pour modifier une image, ce sera sensiblement la même chose que pour l'enregistrement, mais au lieu d'enregistrer une nouvelle image, nous en récupérons une qui est existante à l'aide de **findOne()** puis on applique la méthode **updateOne() pour renseigner les informations à mettre à jour :**

```
//Méthode chargée de modifier un fichier
exports.modifyOneFile = (req, res, next) => {

    const file = new File({
        name: req.file.filename,
        description: req.body.description,
        imageUrl: `${req.protocol}://localhost:3000/uploads/${req.file.filename}`,
        userId: req.body.userId
    });

    File.findOne({_id: req.params.id})
        .then((thing) => {
            if(file.userId == thing.userId){
                File.updateOne({ _id: req.params.id}, { ...file, _id: req.params.id})
                .then(() => res.status(200).json({message : 'Objet modifié!'}))
                .catch(error => res.status(401).json({ error }));
            }
        })
        .catch((error) => {
            res.status(400).json({ error });
        });
};
```

Supprimer une image

Récupérez **l'id de l'image** à supprimer et utilisez **findOne()** pour trouver l'image. Enfin, procédez à la suppression grâce à **deleteOne()**. Ici, nous utilisons non seulement mongoose pour supprimer l'enregistrement en base de données, mais nous utilisons également le module fs qui nous permet de supprimer le fichier qui a été stocké sur le disque lors du téléversement.

```
//Méthode chargée de supprimer un fichier
exports.deleteOneFile = (req, res, next) => {
    File.findOne({ _id: req.body.id})
    .then(file => {
        const filename = file.imageUrl.split('/uploads/')[1];
        fs.unlink(`uploads/${filename}`, () => {
            File.deleteOne({_id: req.body.id})
                .then(() => { res.status(200).json({message: 'Objet supprimé !'})})
                .catch(error => res.status(401).json({ error }));
        });
    }
    )
    .catch( error => {
        res.status(500).json({ error });
    });
};
```

Faites en sorte qu'en empruntant la route / :id en DELETE, multer intervienne pour supprimer l'image dans le répertoire uploads.

Je retiens : l'authentification d'un utilisateur

On utilise une route et un service pour authentifier l'utilisateur, en combinaison avec le système de token JWT pour authentifier un utilisateur tout le long de sa navigation et sécuriser (privatiser) les routes de l'API.
Pour gérer les fichiers provenant de l'utilisateur, on utilise multer. Il permet d'uploader les fichiers. On crée alors une fonctionnalité pour renommer le fichier durant l'upload afin de rendre ce nom unique.
On crée des routes pour exprimer le CRUD du fichier correspondant en base de données.

Déterminez les performances actuelles d'une application

C'est la première étape du processus et pour une API REST, c'est déterminer le nombre de requêtes que notre route d'authentification est capable de satisfaire à la seconde, par exemple.
Il existe de nombreux outils de test de charge comme locust, gatling, artillery ou autocannon.
Autocannon est simple d'utilisation et est généralement un bon choix. Pour l'installer, tapez la commande :
npm install autocannon -g.

Pour vous servir d'Autocannon, voici le format de la commande, vous remarquerez tout de suite que vous pouvez fournir une série d'options, nous allons en tirer parti de manière à fournir un body par exemple :

autocannon [options] URL

Les options dont vous allez pouvoir vous servir sont les suivantes, lisez bien chaque ligne de cette documentation, l'option se trouve à gauche, et sa description se trouve toujours sur la même ligne, à droite :

```
-c/--connections NUM : Le nombre de connexions concurrentes. Par défaut: 10.
-d/--duration SEC : Le temps d'exécution d'autocannon en seconde. Par défaut: 10.
-m/--method METHOD :  La méthode HTTP à utiliser. Par défaut: 'GET'.
-b/--body BODY : Le corps de la requête.
-i/--input FILE : Le chemin d'un fichier contenant le corps de la requête.
```

Pour effectuer le test de charge avec autocannon, lancez l'API et testez la route **localhost :3000/users/authenticate** en fournissant un email et un mot de passe :
autocannon -c 100 -m POST -b '{"email":"john@xxx.com","password":"1236666"}' -H Content-Type=application/json localhost:3000/users/authenticate

```
jeanm@Jarvis MINGW64 ~/Desktop/api
$ autocannon -c 100 -m POST -b '{"email":"john@xxx.com","password":"1236666"}' -
ticate
Running 10s test @ http://localhost:3000/users/authenticate
100 connections

running [========            ] 40%
```

Puis, le rapport s'affiche dans le terminal :

```
jeanm@Jarvis MINGW64 ~/Desktop/api
$ autocannon -c 100 -m POST -b '{"email":"john@xxx.com","password":"1236666"}'
ticate
Running 10s test @ http://localhost:3000/users/authenticate
100 connections
```

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Ma
Latency	426 ms	1818 ms	2020 ms	2031 ms	1707.83 ms	466.88 ms	66

Stat	1%	2.5%	50%	97.5%	Avg	Stdev	Min
Req/Sec	48	48	54	59	53.4	3.53	48
Bytes/Sec	30.3 kB	30.3 kB	34.1 kB	37.2 kB	33.7 kB	2.22 kB	30.3

```
Req/Bytes counts sampled once per second.
# of samples: 10

634 requests in 10.1s, 337 kB read
```

634 requêtes ont été envoyées par nos 100 connexions concurrentes (**option -c**) durant 10 secondes, soit environ 63 requêtes par seconde.

Identifiez les freins dans le programme

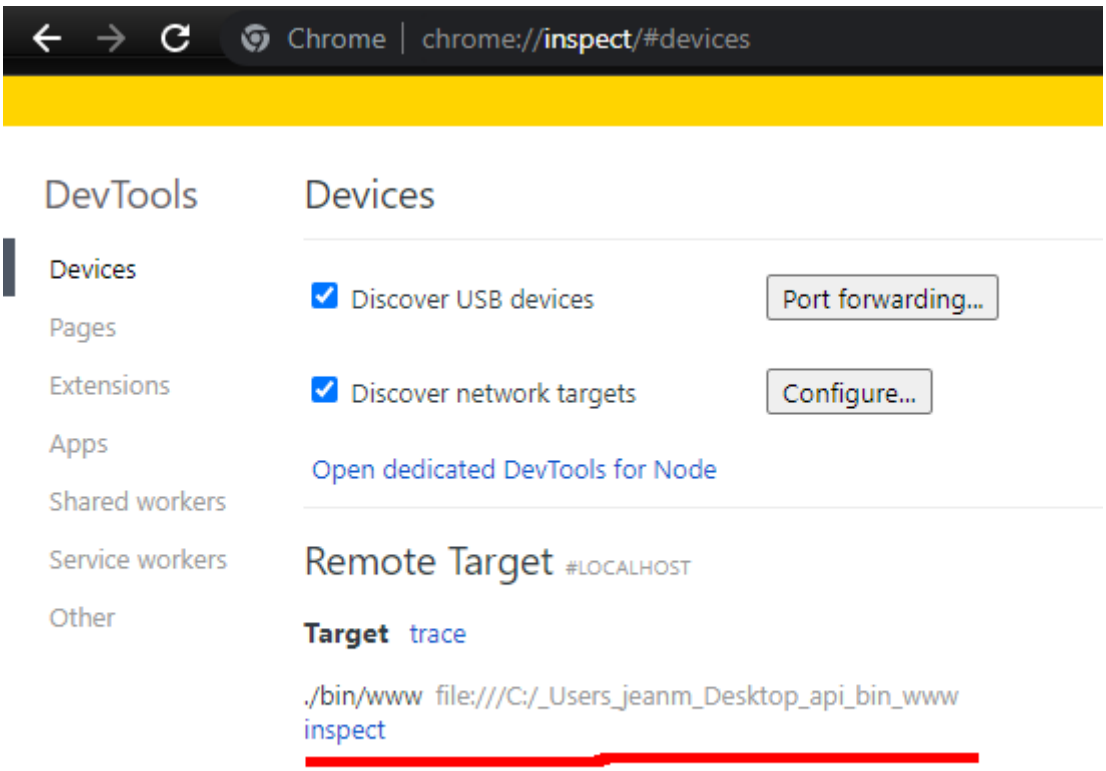
La seconde étape du processus d'optimisation consiste à identifier les freins.
Plusieurs outils existent comme **Profiler V8** ou **Chrome DevTools**.

Chrome DevTools

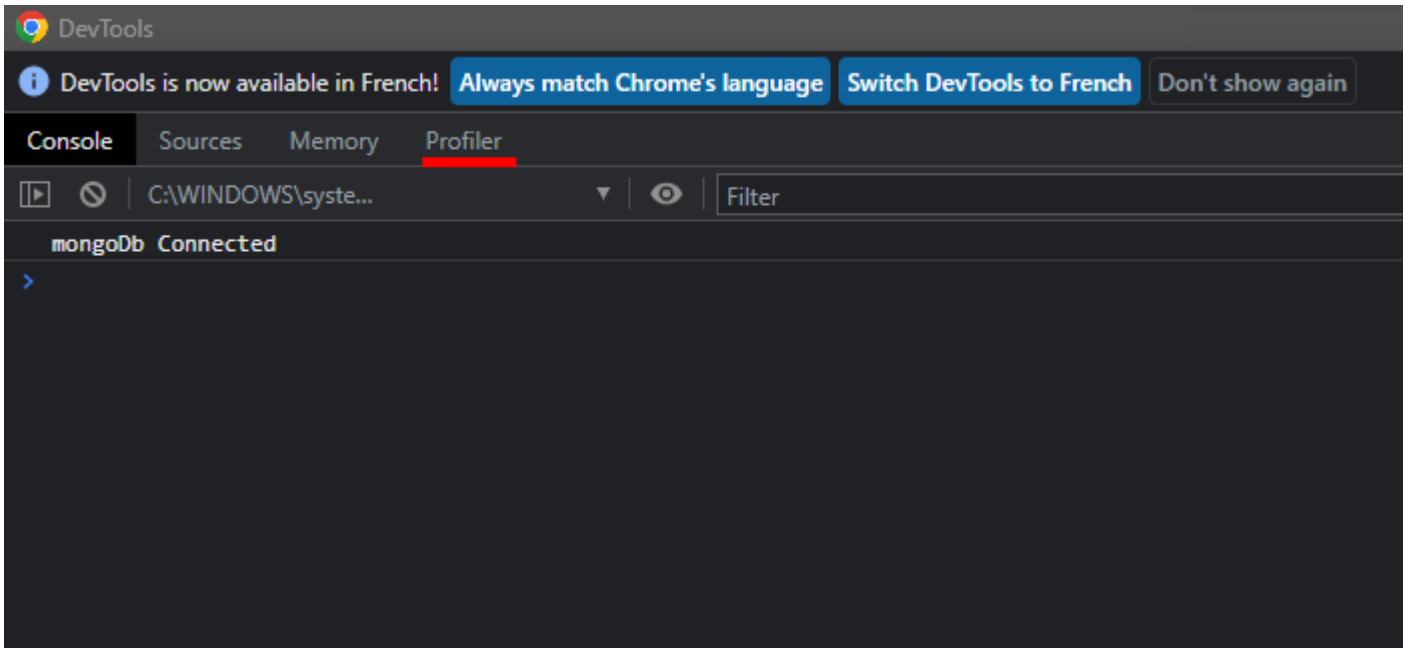
Pour utiliser Chrome DevTools, il faut lancer votre api avec l'option **–inspect**, ce qui permettra au navigateur, à partir de l'URL **chrome://inspect**, de proposer des outils, notamment de profilage. Aussi, adaptez la commande **start** en y ajoutant l'option **--inspect** dans le fichier **package.json.**

```
"scripts": {
  "start": "env-cmd -f ./env/.env nodemon --inspect ./bin/www",
```

Accédez à la page. Vous devriez voir votre application apparaître sur l'interface d'inspection :

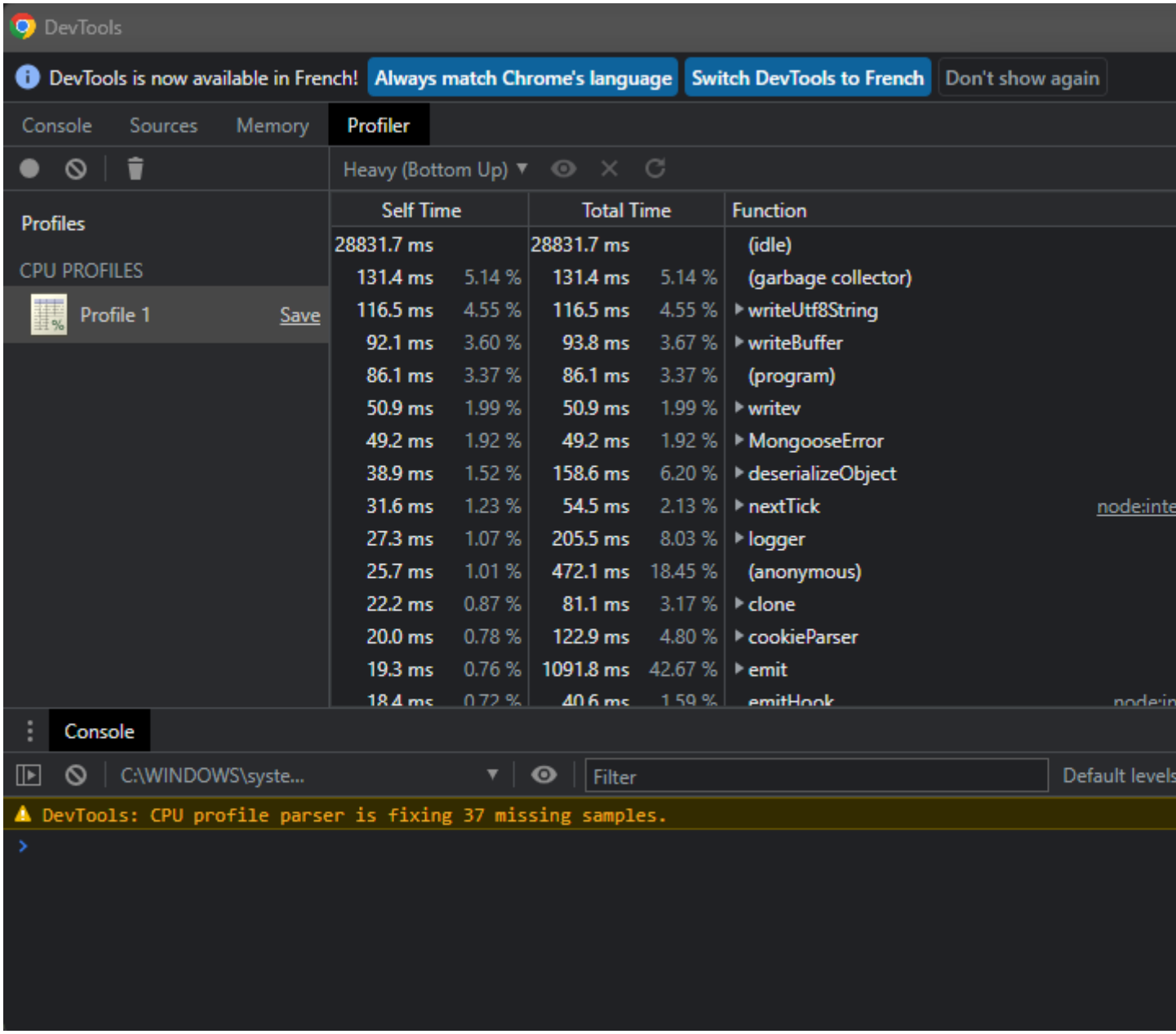


Cliquez sur le lien **inspect** de l'application (image ci-dessus) pour accéder au **profiler**. En réalité, cela ouvre la fenêtre dédiée au profilage des application **Node.Js**. Pour accéder au profiler sur cette fenêtre, cliquez sur l'onglet **profiler** (image ci-dessous).

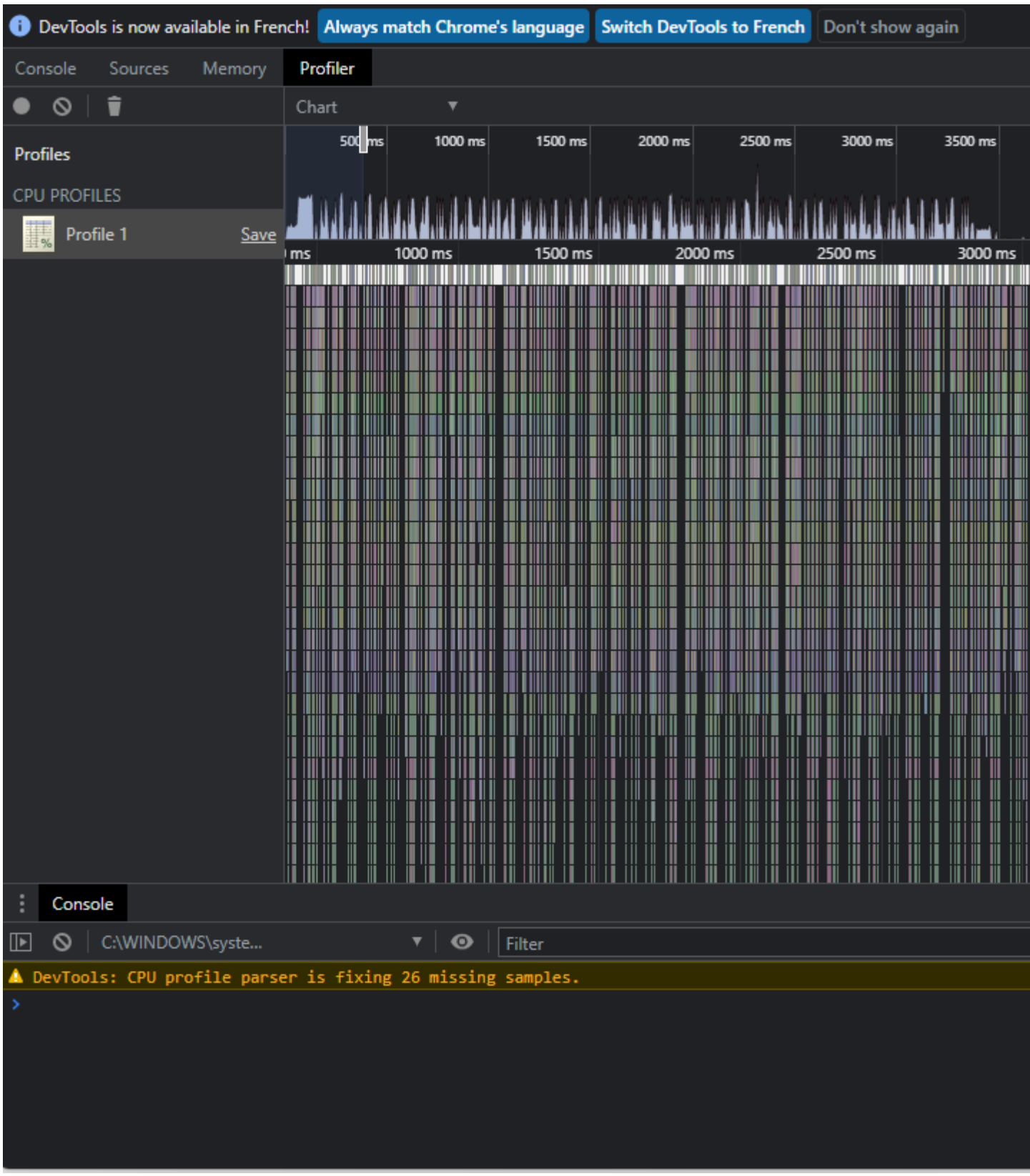


Pour démarrer le **profiling CPU**, cliquez sur le bouton **Start** et lancez la commande autocannon à nouveau. Une fois le processus autocannon achevé, appuyez sur le bouton stop du chrome devTools et ouvrez le fichier de journalisation.
Vous êtes en mesure d'analyser la consommation de temps du CPU pour votre application en utilisant une des trois vues :

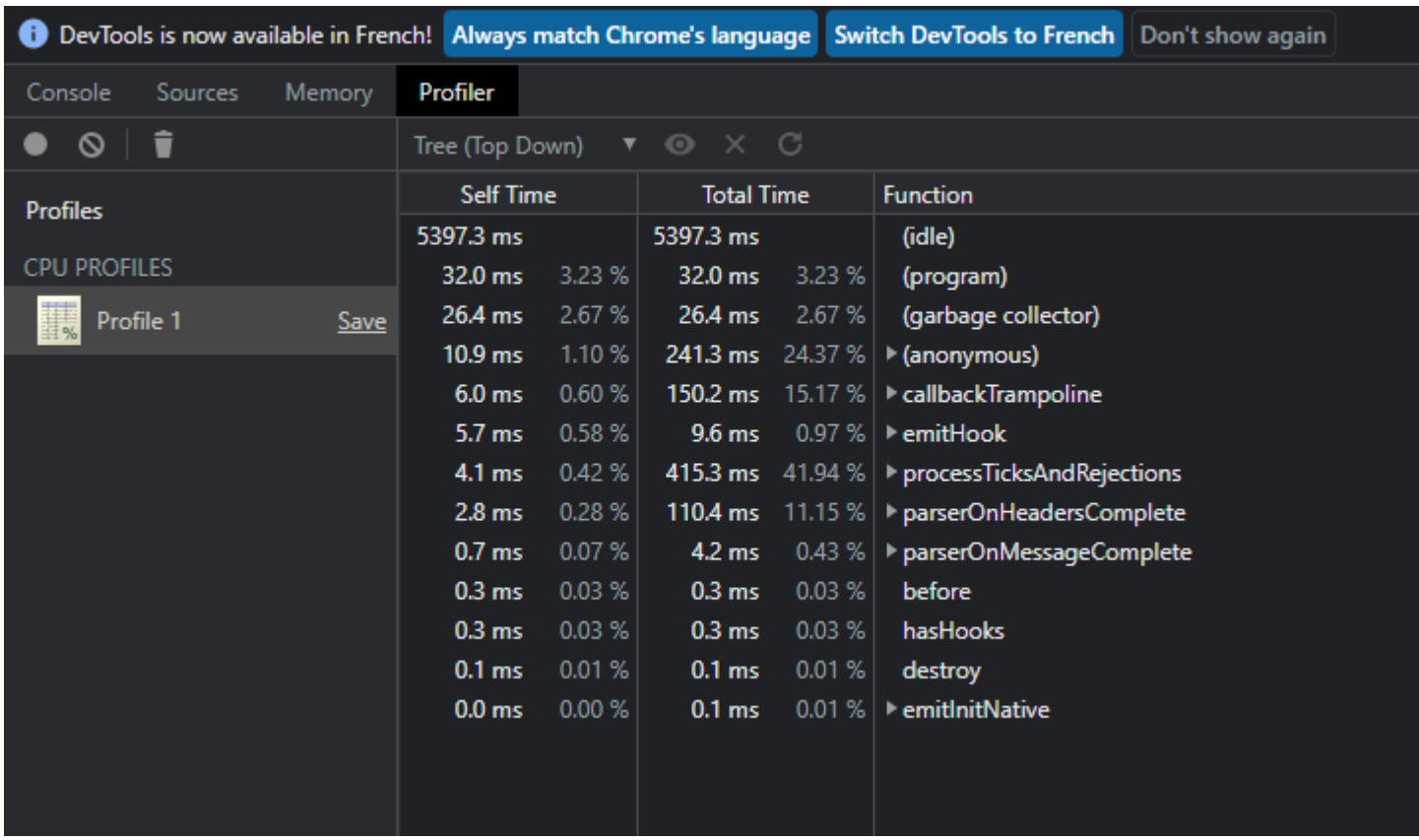
- **Heavy (Bottom Up)** : Cette vue permet de voir quelles fonctions ont eu le plus d'impact sur les performances et d'examiner la pile d'appels (**call stack**). Elle est découpée en trois parties : **Self time** (le temps cumulé passé dans la fonction), **Total time** (le temps total cumulé passé dans la fonction - fonctions appelées comprises) et **Function** (le nom de la fonction).



- **Chart** : C'est une représentation visuelle des processus en cours d'exécution sur le CPU.



- **Tree (Top Down)** : Cette vue permet de voir l'arborescence des appels de fonctions avec les points d'entrée d'exécution.



Les graphes de Chrome devTools sont pratiques mais ne permettent pas de visualiser directement les fonctions les plus consommatrices. C'est ce que permettent en revanche les **flamegraphs.**

Flamegraph

Pour générer un flamegraph, utilisez **ClinicJs**. Installez-le avec :
npm install -g clinic.

Clinic fourni trois outils et notamment **flame**. Pour l'utiliser :
clinic flame [options] -- node ./bin/www
La commande **clinic** ne fonctionne qu'avec la commande **node**.

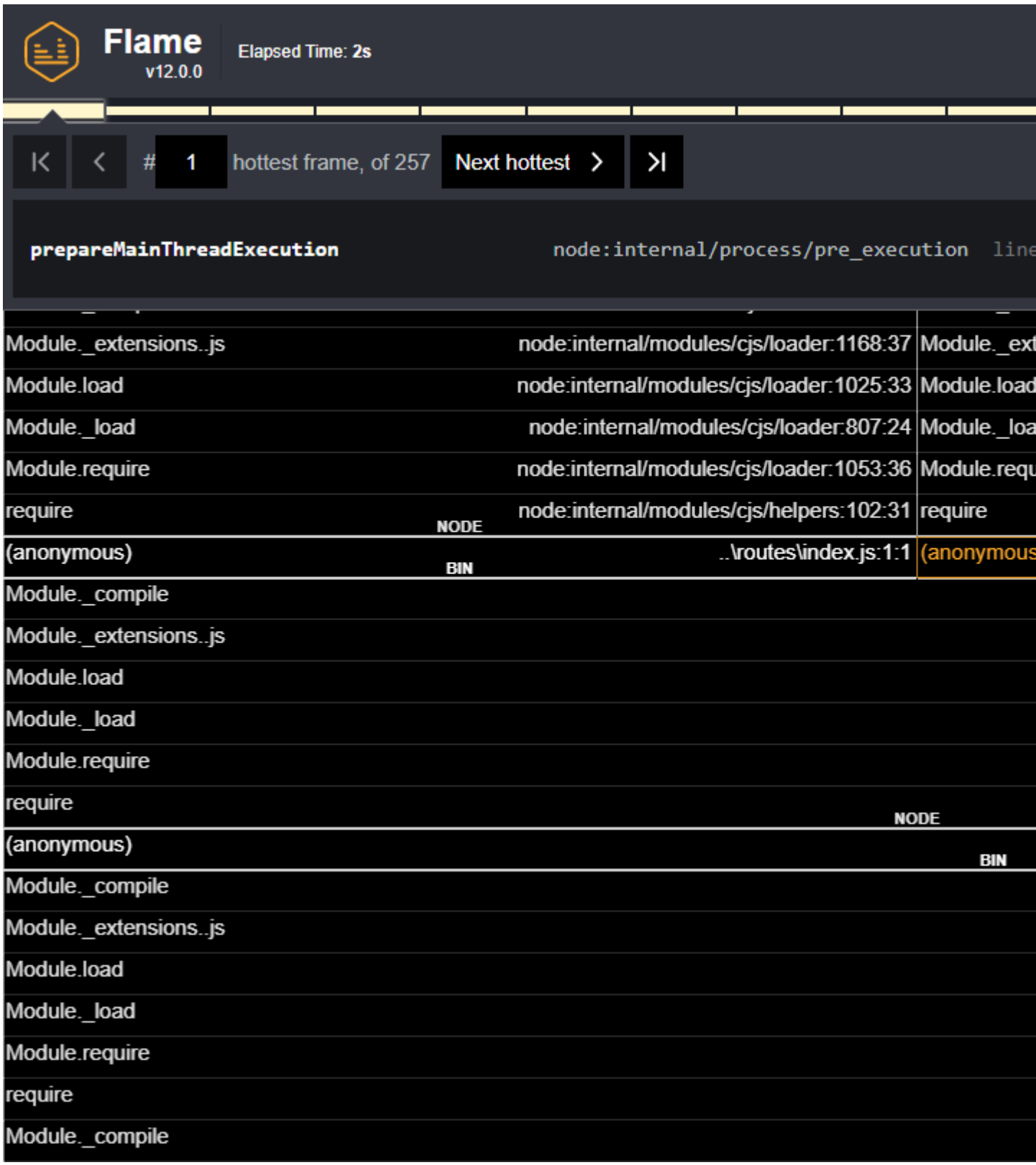
Utilisez les options suivantes :

```
--autocannon
    Permets de lancer autocannon en lui passant ses options entre [ ].
--on-port
    Permets de lancer une commande une fois que l'application est en écoute sur un port.
```

Lancez la commande :

```
jeanm@Jarvis MINGW64 ~/Desktop/api
$ clinic flame --autocannon [-c 100 -m POST -b '{"email":"john@xxx.com","passw
000/users/authenticate] -- node ./bin/www
```

Après l’analyse, accédez à une interface :



Vous visualisez parfaitement les fonctions qui prennent le plus de temps CPU. Cela vous permet d'optimiser les algorithmes de vos fonctions les plus consommatrices.
Un conseil : utilisez le plus possible **les versions asynchrones des fonctions** car **elles ne bloquent pas l'event loop** (la boucle d'évènements node).

Pour aller plus loin : flamebearer

Il existe d'autres outils de génération de Flamegraph comme **flamebearer**.

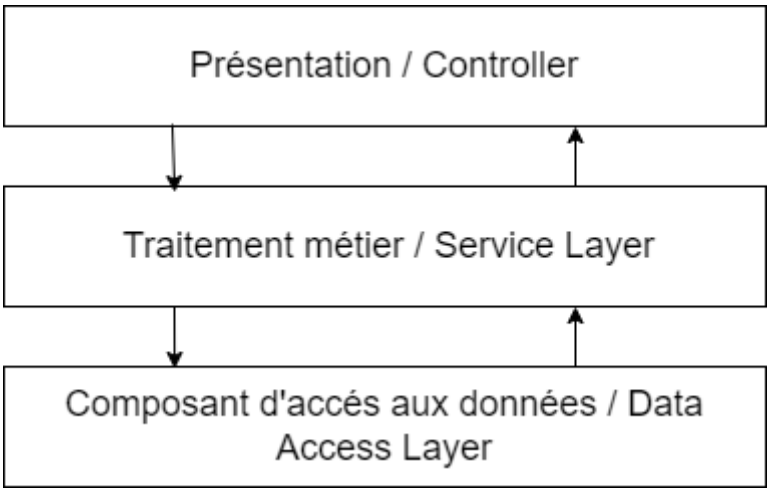
Concevez une architecture adaptée

Le principe de séparation des préoccupations

La séparation des préoccupations, ou Separation of concerns, est un principe de segmentation d'un programme informatique dans lequel chaque segment de code ne contient que les informations nécessaires à sa fonctionnalité.
Ce principe est appliqué dans l'**architecture d'une application (design pattern)** générée avec **express-generator**. Les **routes**, les **modèles**, les **vues**, les **services** et les **middlewares** sont chacun dans leurs répertoires respectifs.

L'architecture à trois couches ou trois tiers

L'idée est d'utiliser le principe de séparation des préoccupations en éloignant la logique métier des routes de l'API. Les trois couches s'empilent ainsi :
La couche de **présentation des données** : l'affichage et le dialogue avec l'utilisateur grâce aux gestionnaires de routes (**controllers**) et vues (**views**).
La couche de **traitement métier** : les règles de gestion et la logique applicative avec les **services** et les **middlewares**).
La couche d'**accès aux données persistantes** : les données qui sont destinées à être conservées dans le temps (**data access components** et **models**).



Les couches communiquent entre elles en s'appuyant sur un modèle d'échange. Chaque couche ne communique qu'avec ses voisins immédiats.
Les objectifs sont les suivants :

- Alléger le poste de travail client car le traitement se fait côté serveur et une seule couche s'occupe du client.
- Offrir une portabilité sur toutes les plates-formes (serveurs, clients, langages).
- Mieux répartir la charge entre les différents serveurs d'application

Définissez l’architecture d’une application Node.Js

Elle contiendra presque toujours un répertoire **src** à la racine, accompagné des répertoires **node_modules** et des fichiers **package.json** et **app.js**. Dans ce répertoire source **src**, vous déclarez les répertoires **models**, **services**, **middlewares** et **routes** permettant de respecter l’architecture 3-tiers et le principe de séparation des préoccupations.

Utilisation d’une couche de service

C’est l’endroit de votre logique métier, ce que fait l’application, en somme. Dans cette couche, il ne doit exister, dans l’idéal, aucune requête à la base de données. Quelques principes à respecter dans cette couche :
Séparer le code du routeur express.js.
Ne pas transmettre l’objet **req** ou **res** à **la couche service.**
Ne renvoyer aucun élément lié à la couche de transport HTTP, tel qu’un code d’état ou des en-têtes de la **couche de service**.

Architecture Pub/Sub

Ce modèle d’architecture étend l’architecture 3-tiers classique. Imaginez un **endPoint** (route) de votre API qui crée un utilisateur. Ce dernier voudra peut-être appeler un service tiers comme un service d’analyse. Tôt ou tard, cette opération fera plusieurs choses en une seule fonction de centaines de lignes de code. Cela va à l’encontre du **principe de responsabilité unique**.
L’architecture Pub/Sub propose de séparer les responsabilités dès le départ. Le principe est d’émettre un évènement (**eventEmitter**) et de laisser la responsabilité à des (**eventListeners**) de poursuivre le traitement.

Je retiens : l'optimisation d'une application

L’optimisation d’une application Node.js se découpe en trois étapes, le test de charge, la détection des freins et l’optimisation du code des fonctions incriminées.

Vous utilisez un outil de test de charge comme autocannon pour déterminer les performances actuelles de votre application.

Pour analyse les performances vous utilisez les Chrome Dev Tools et pour pousser plus loin cette analyse en ciblant les freins vous utilisez un Flamegraph.

Dans une application, Le code est séparé en couches : la présentation, le traitement métier (service) et les composants d’accès aux données. Cela s’appelle l’architecture à trois couches ou trois tiers (écrit aussi 3-tiers).

Appliquez la persistance côté client

Dans le cadre d’une application web, les échanges client-serveur nécessitent de pouvoir pérenniser une partie des informations sur la machine cliente.

Cookie

C’est un fichier stocké dans la machine d’un utilisateur et associé à un domaine web. Ce fichier est automatiquement renvoyé lors de contacts ultérieurs. Les cookies peuvent servir à mémoriser un identifiant client ou le contenu courant d’un panier d'achat, par exemple.
Le cookie se sécurise avec des options comme **secure**, qui permet de limiter l’envoi du cookie uniquement via HTTPS, et **samesite**, qui empêche le navigateur d’envoyer un cookie lors d’une requête provenant d’un autre site. **samesite="strict"** signifie que le cookie ne doit jamais être envoyé si l’utilisateur arrive sur le domaine depuis un autre site.

Déclarer un cookie avec JS

```
//définition de la date d'expiration du cookie. Ici 86400000ms = 1 jour
let date = new Date(Date.now() + 86400000);
// Conversion de la date au format UTC
date = date.toUTCString();

//Crée ou met à jour un cookie 'user'
document.cookie = 'user=Jim; path=/; domain=jim.com; samesite="strict"; secure; expires=

//Supprimer le cookie peut se faire en lui passant une date d'expiration passée
document.cookie = 'user=; path=/; expires=Thu, 01 Jan 1970 00:00:00 UTC';
```

Express dispose d’un module cookie-session. Il stocke les données de session sur le client dans un cookie.

```
var cookieSession = require('cookie-session')
var express = require('express')

var app = express()

app.use(cookieSession({
    name: 'session',
    keys: [/* secret keys */],

    // Cookie Options
    maxAge: 24 * 60 * 60 * 1000 // 24 hours
}))
```

Il est possible de visualiser l’ensemble des cookies stockés sur votre ordinateur via le navigateur en passant par la console de développement. Ouvrez la console et tentez de visualiser les cookies enregistrés.

Le Window.sessionStorage

Un objet **sessionStorage** est valide pour la session de navigation en cours et pour les pages du même domaine. Les données enregistrées dans sessionStorage ont une durée vie limitée et expirent à la fin de la session de navigation (lorsque vous fermez l'onglet de la page courante).
Le module express-session stocke un identifiant de session sur le client dans un cookie et stocke les données de session sur le serveur.

```
var app = express()

app.use(session({
    secret: 'mysecret',
    resave: false,
    saveUninitialized: true,
    cookie: { secure: true }
}))
```

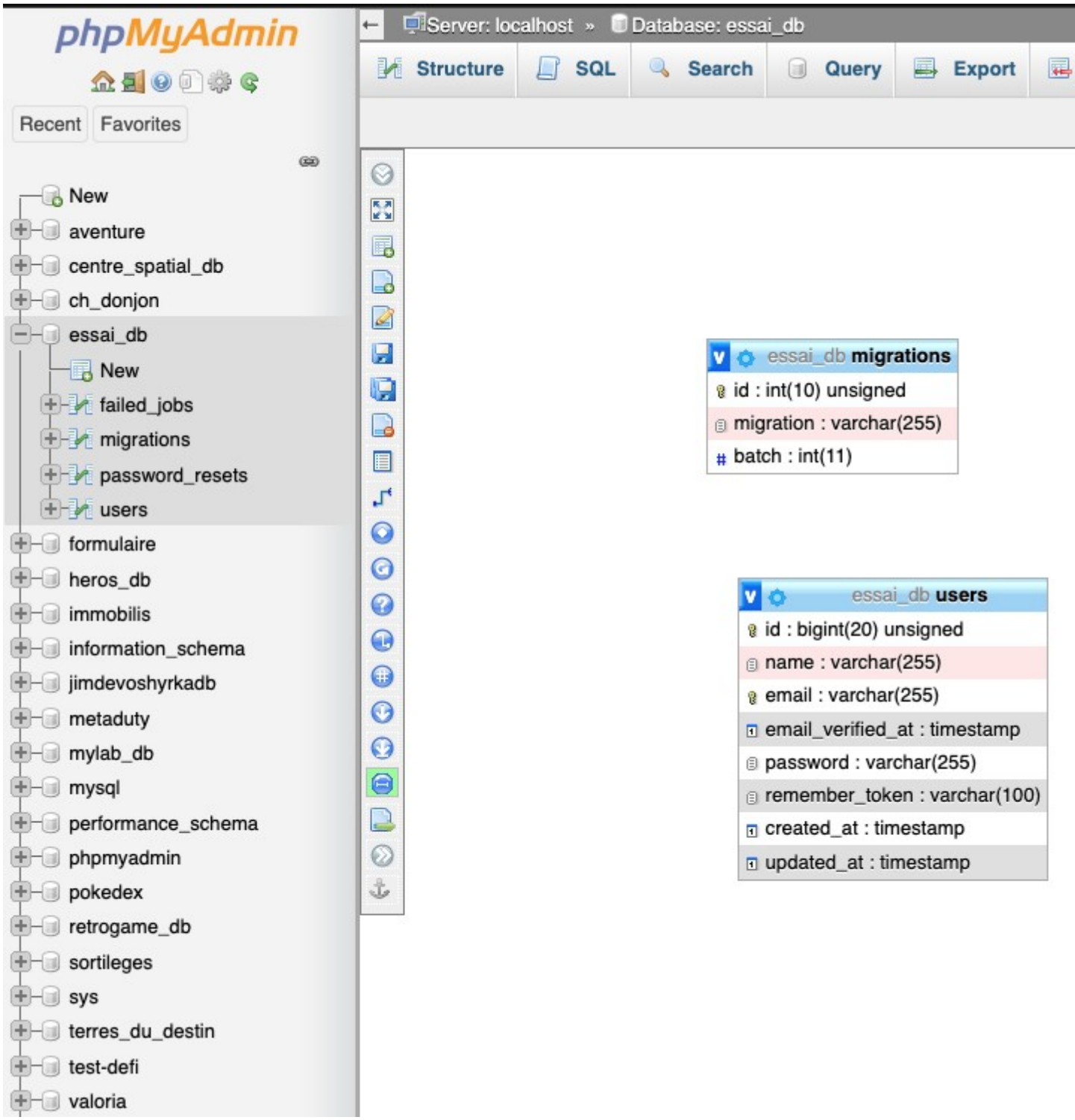
Le Window.localStorage

Le **localStorage** est voisin du **sessionStorage**. Cependant, les données stockées dans le localStorage n'ont pas de **délai d'expiration**. Node dispose d'un module node-localstorage.

Appliquez la persistance côté serveur

La persistance côté serveur se fait essentiellement avec des bases de données. Une base de données est un ensemble d'informations organisées de manière à être facilement accessibles, gérées et mises à jour.

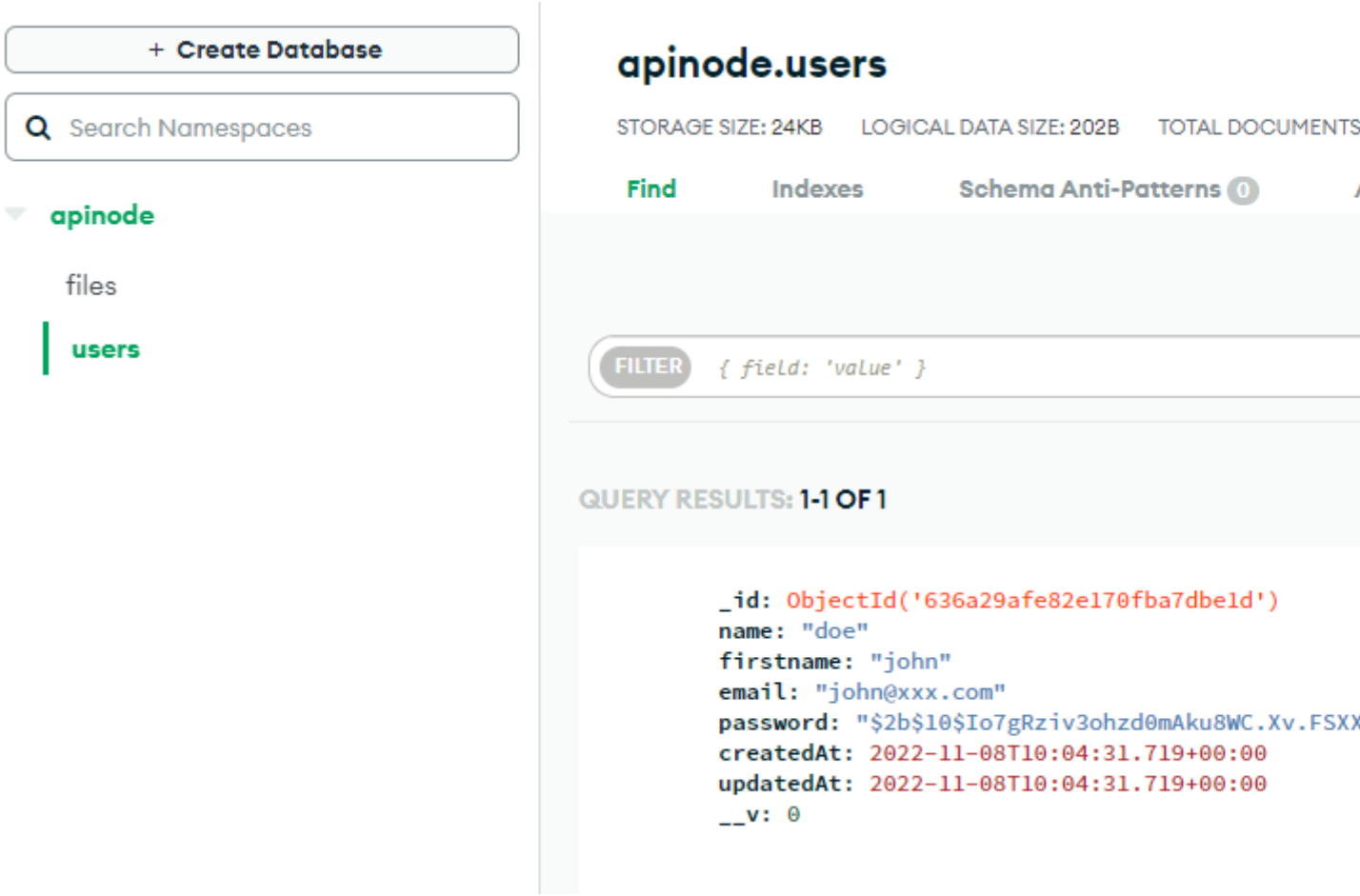
Les bases de données relationnelles SQL



Les bases de données relationnelles sont composées d'un ensemble de tables qui peuvent être accessibles et reconstruites de différentes manières. Le **langage de requête SQL** est l'interface qui permet d'interroger et manipuler les données contenues. On peut citer **SQL Server** ou le serveur de base de données **MySQL** / **MariaDB** sous Linux. Pour les applications mobiles, il existe également **sqlite**.

Les bases de données NoSQL

C’est ce que vous avez expérimenté comme type de base de données en utilisant **mongoDB**. L’ère du **BigData** s’appuie sur des bases de données NoSQL. Ce sont des bases de données « **non relationnelles** ». Les jointures ne sont plus nécessaires et le scaling est facilité.



Pour aller plus loin : documentation technique

Découvrez les documentations de MySQL, SQL Server, Solr, ou mongoDB.

Je retiens : la persistance des données

Côté client, la persistance se fait grâce à des cookies de session ou le localStorage.
Côté serveur, elle se fait grâce aux bases de données.